Artificial Intelligence for Science

南方科技大学 SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

地球与空间科学系 DEPARTMENT OF EARTH AND SPACE SCIENCES

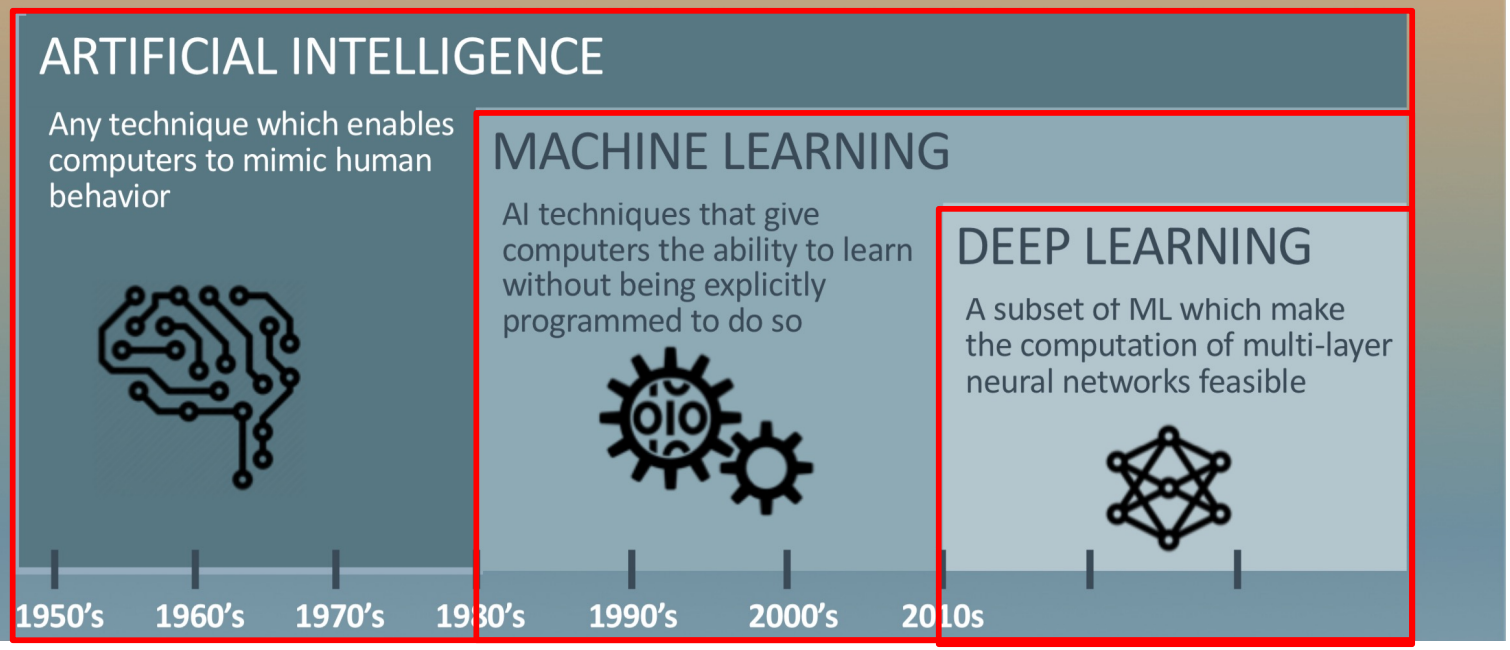
Lesson 17: Review: What have we learned?

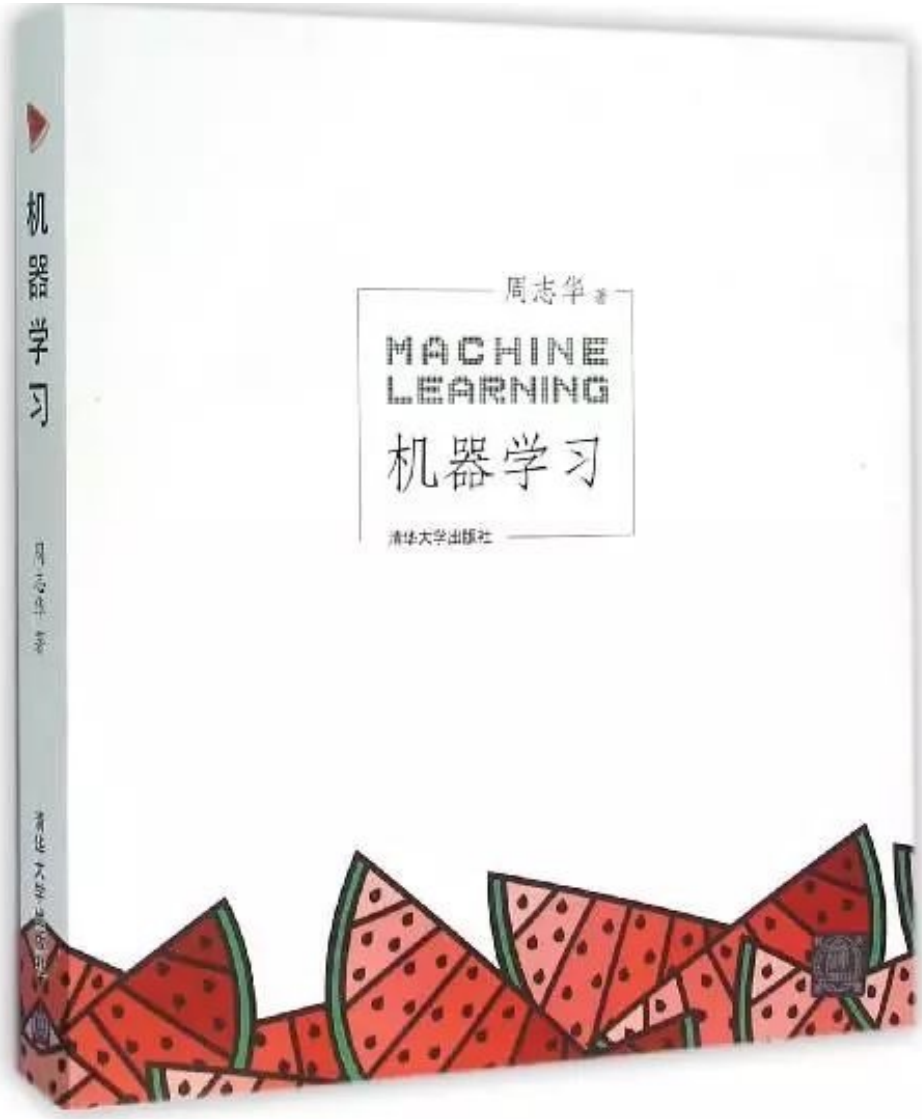
CHEN Kejie 陈克杰（chenkj@sustech.edu.cn）

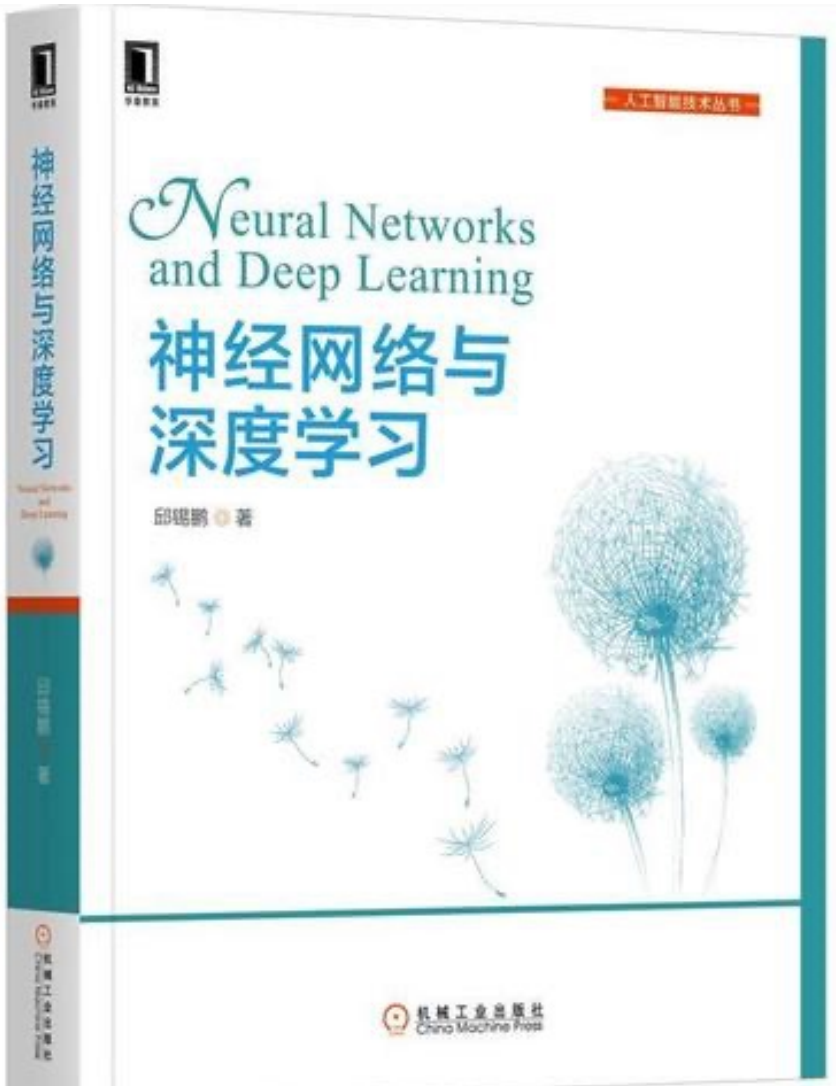
3-Jun-26

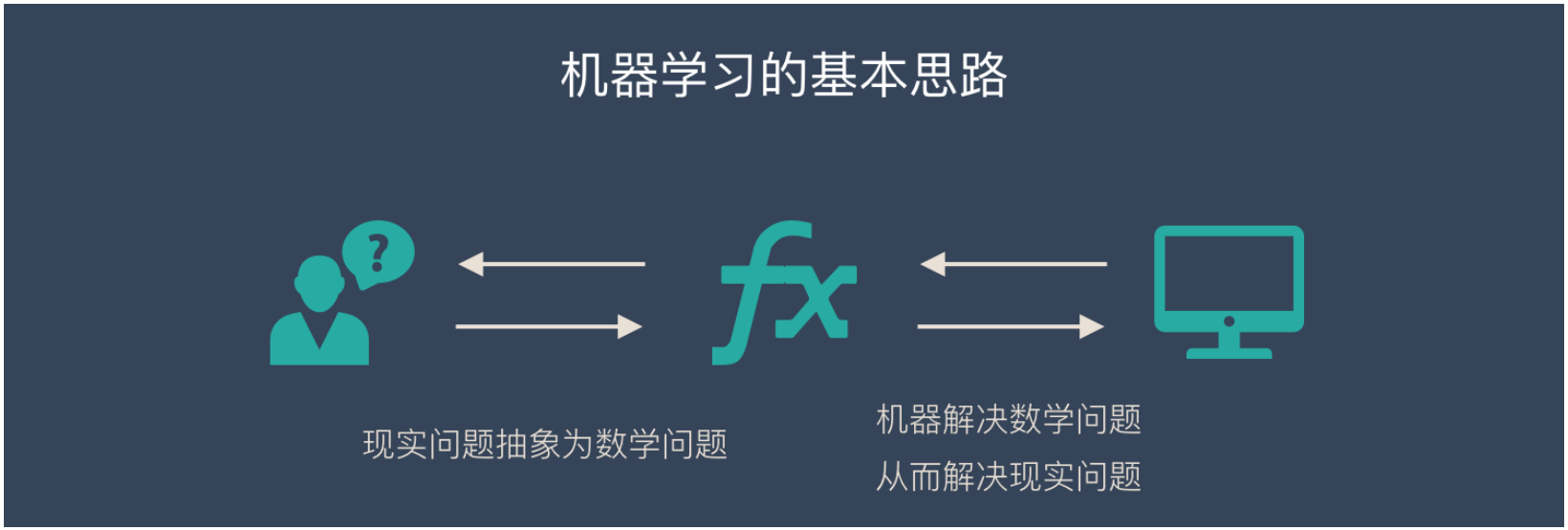
ARTIFICIAL INTELLIGENCE

IS NOT NEW




机器学习
周志华 著
MACHINE LEARNING
机器学习
清华大学出版社

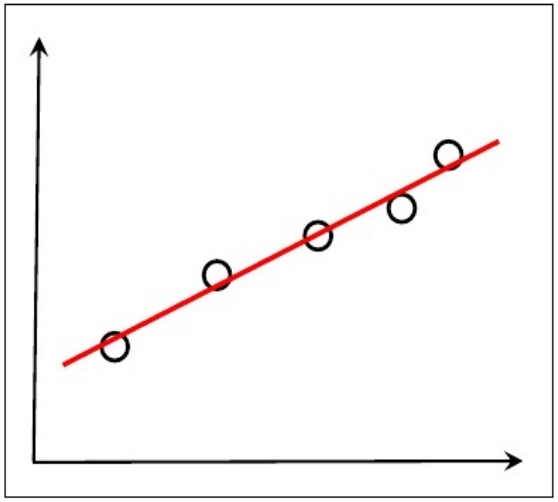

人工智能技术丛书
神经网络与深度学习
Neural Networks and Deep Learning
神经网络与深度学习
邱锡鹏 著
机械工业出版社
China Machine Press

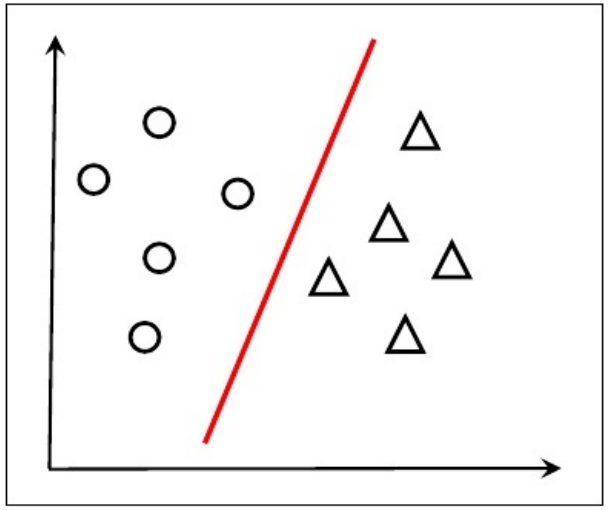


模型评估和选择:

训练误差，泛化误差
留出法，交叉验证
查准率，查全率
P-R曲线
AUC ROC
……

A **linear model （线性模型）** attempts to learn a function that makes predictions by a linear combination of properties.





$$f(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + \ldots + w_d x_d + b$$

$$f(\boldsymbol{x}) = \boldsymbol{w}^{\mathrm{T}} \boldsymbol{x} + b$$

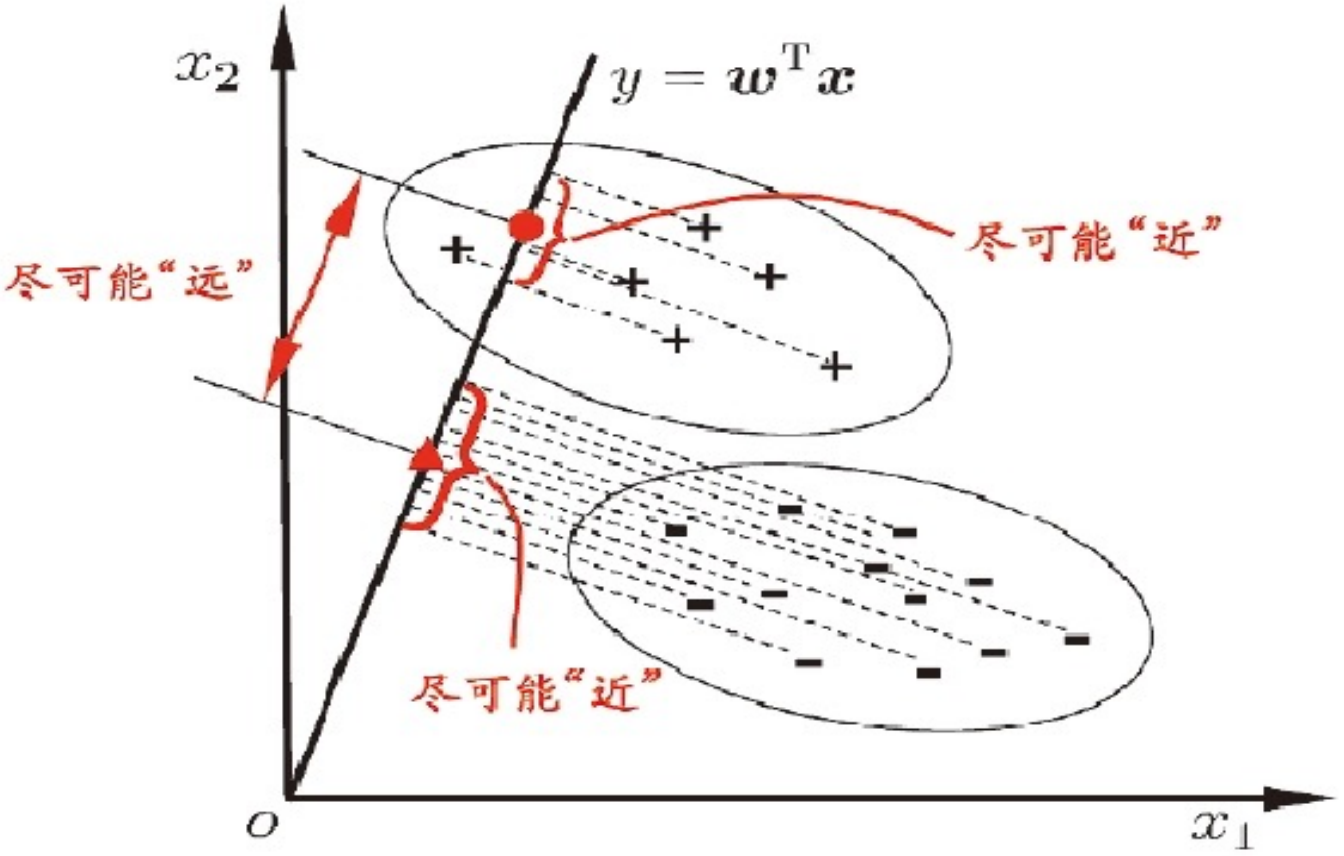
广义线性模型，如逻辑回归（logistic regression）

$$y = \frac{1}{1 + e^{-z}}$$

$$y = \frac{1}{1 + e^{-(\boldsymbol{w}^{\mathrm{T}} \boldsymbol{x} + b)}}$$

$$\ln \frac{y}{1 - y} = \boldsymbol{w}^{\mathrm{T}} \boldsymbol{x} + b$$

Linear Discriminant Analysis （线性判别分析）



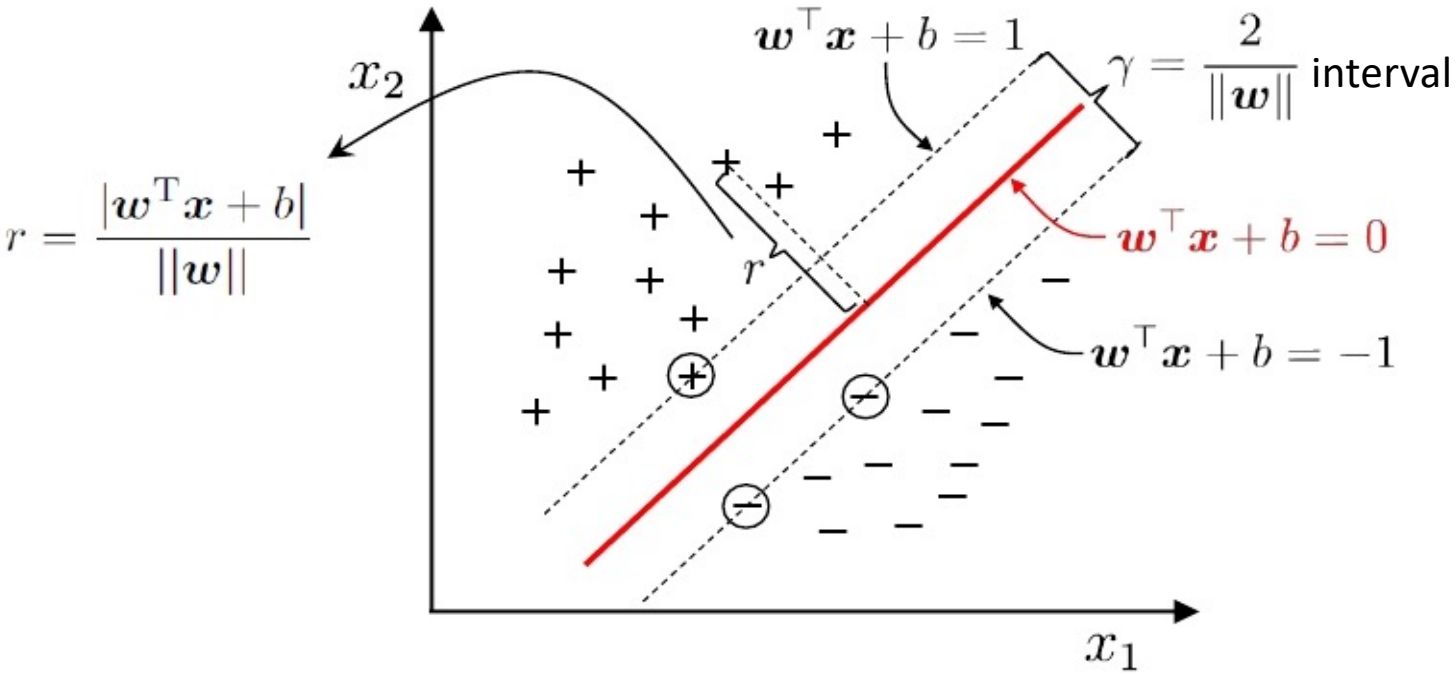
Fisher [1936]，又称Fisher判别分析

Because the sample is projected onto a line (low dimensional space), it is also considered as “supervised dimensionality reduction.

Support Vector Machines
(支持向量机)

Hyperplane equation:

$$\boldsymbol{w}^\top \boldsymbol{x} + b = 0$$



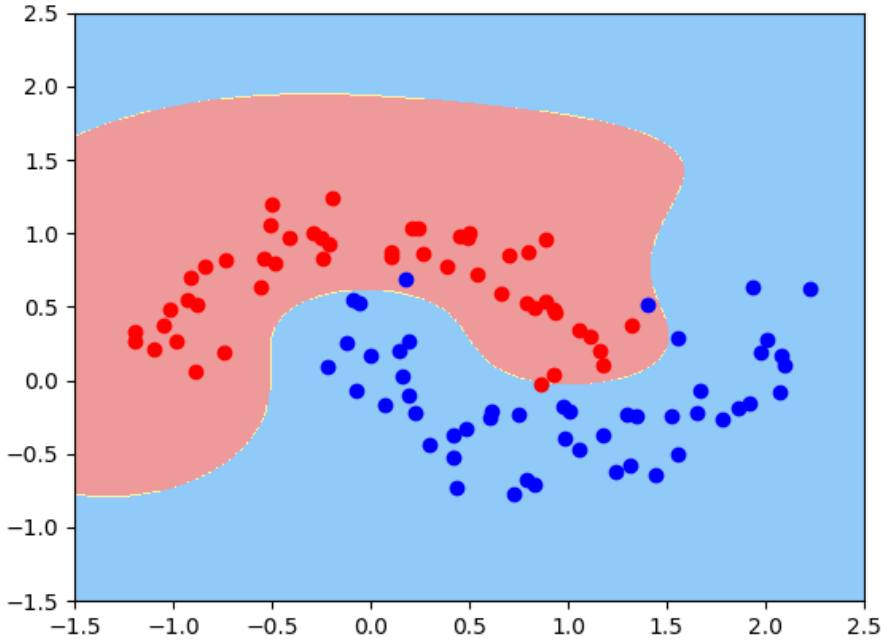
Kernel function (核函数)

Basic idea: design kernel function

$$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^{\mathrm{T}} \phi(\boldsymbol{x}_j)$$

Bypassing the difficulty of explicitly considering feature mapping and calculating high-dimensional inner products

名称	表达式	参数
线性核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{x}_i^{\mathrm{T}} \boldsymbol{x}_j$	
多项式核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\boldsymbol{x}_i^{\mathrm{T}} \boldsymbol{x}_j)^d$	$d \geqslant 1$ 为多项式的次数
高斯核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2}{2\sigma^2}\right)$	$\sigma > 0$ 为高斯核的带宽(width)
拉普拉斯核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|}{\sigma}\right)$	$\sigma > 0$
Sigmoid 核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \tanh(\beta \boldsymbol{x}_i^{\mathrm{T}} \boldsymbol{x}_j + \theta)$	tanh 为双曲正切函数, $\beta > 0$, $\theta < 0$



Decision Tree (决策树)

The value of discrete attribute a: $\{a^1, a^2, \ldots, a^V\}$ 非连续属性

Dv: The sample set of D with a value of a = a^v

The information gain obtained by dividing the data set D with attribute a is:

$$\mathrm{Gain}(D, a) = \mathrm{Ent}(D) - \sum_{v=1}^{V} \frac{|D^v|}{|D|} \mathrm{Ent}(D^v)$$

Information entropy before partition

Information entropy after partition

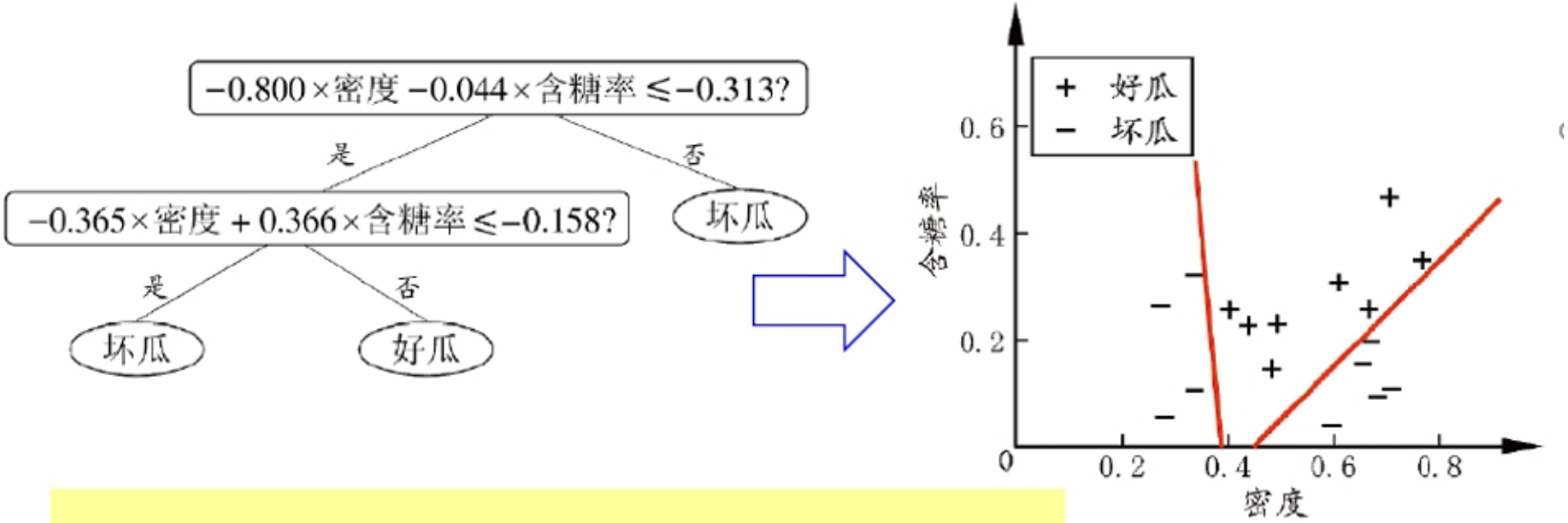
Used in ID3 Algorithm (Iterative Dichotomiser 3)

The weight of the v branch: the more samples, the more important

Multivariate decision tree (多变量决策树)

Multivariate decision tree: each non-leaf node considers not only one attribute

For example, "oblique decision tree" is not to find the optimal partition attribute for each non-leaf node, but to build a linear classifier



More complex "hybrid decision trees" can even embed neural networks or other nonlinear models in the nodes

A Discriminative model models the **decision boundary between the classes**. A Generative Model explicitly models the **actual distribution of each class**. In final both is predicting the conditional probability P(Animal | Features). But both models learn different probabilities.

In Math,
Training classifiers involve estimating f: X -> Y, or P(Y|X)

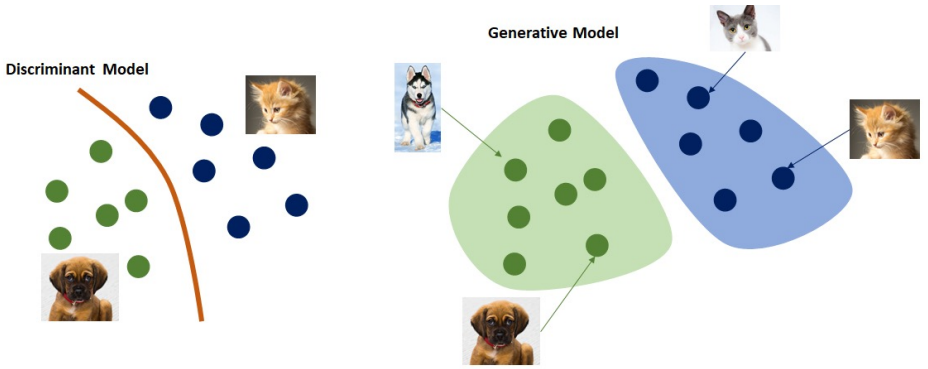
Discriminative Classifiers

•Assume some functional form for P(Y|X)
•Estimate parameters of P(Y|X) directly from training data

Generative classifiers

•Assume some functional form for **P(Y), P(X|Y)**
•Estimate parameters of **P(X|Y), P(Y)** directly from training data
•Use Bayes rule to calculate **P(Y |X)**

Discriminative vs. Generative (判别式 vs 生成式)



Discriminative model

Idea: model $P(c \mid \boldsymbol{x})$ directly
Representative:
- Decision tree
- BP neural network
- SVM

Generative model

Idea: model the joint probability distribution $P(\boldsymbol{x}, c)$ first, and then obtain

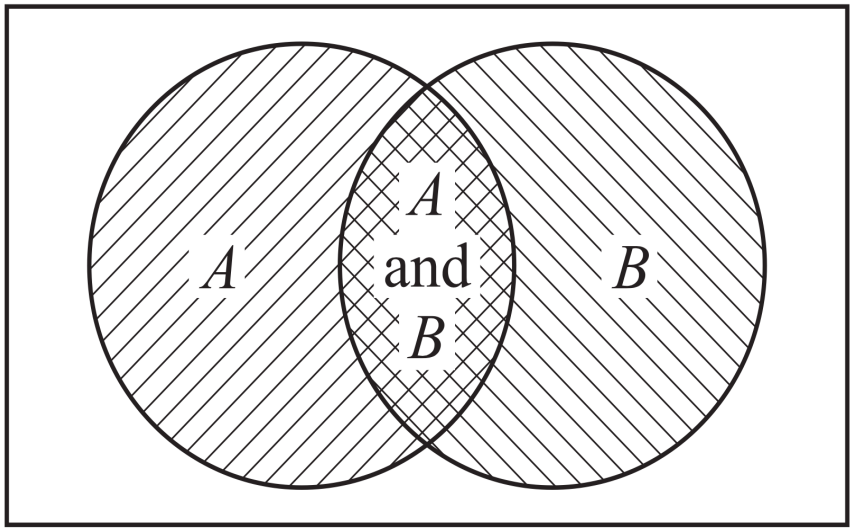
$P(\boldsymbol{x}, c)$ $P(c \mid \boldsymbol{x}) = \frac{P(\boldsymbol{x}, c)}{P(\boldsymbol{x})}$

Representative: Bayesian classifier

Bayes' theorem (贝叶斯定理)

- ✓ **Let us look at a Venn diagram to visualize 2 events A and B. The areas of the 2 circles represent the event probabilities. The overlapping area is the event {A AND B} (i.e., $A \cap B$); the entire surface is the event {A OR B} (i.e., $A \cup B$)**

1. **What is the probability $P(A \cup B)$?**
2. **What is the probability $P(A \cap B)$?**



- ➢ $\boldsymbol{P(A \cup B) = P(A) + P(B) - P(A \cap B)}$**. We need to subtract the intersection area to avoid double-counting**
- ➢ $\boldsymbol{P(A \cap B) = P(A)P(B|A)}$
- ➢ **Observe that we can reverse probabilities, so we also have $\boldsymbol{P(A \cap B) = P(B)P(A|B)}$**

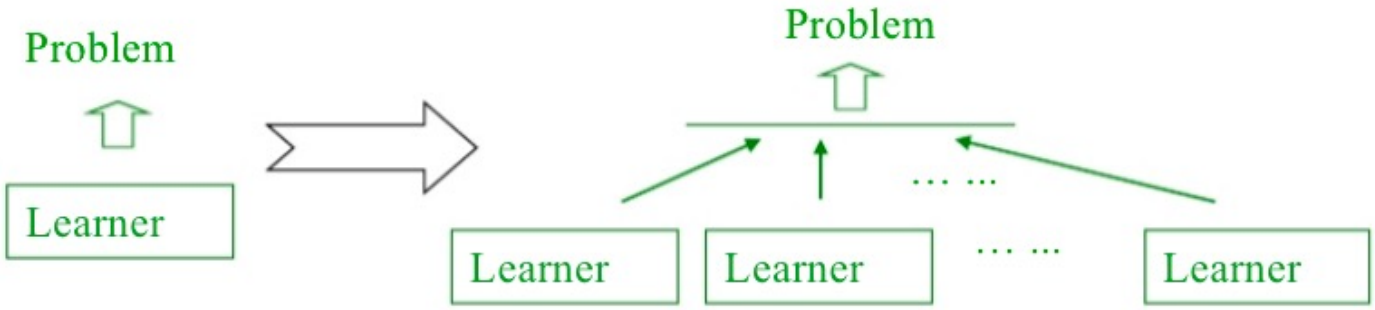
► **Bayes' Theorem**

$$\boldsymbol{P(A|B) = P(B|A)\, P(A) \Big/ P(B)}$$

Ensemble Learning （集成学习）

三个臭皮匠（弱学习器）
顶个诸葛亮

- Serialization method
  - AdaBoost [Freund & Schapire, JCSS97]
  - GradientBoost [Friedman, AnnStat01]
  - LPBoost [Demiriz, Bennett, Shawe-Taylor, MLJ06]
  - … …
- Parallelization method
  - Bagging [Breiman, MLJ01]
  - Random Forest
  - Random Subspace [Ho, TPAMI98]
  - … …



Clustering

K-means:

K-means clustering aims to partition *n* observations into *k* clusters in which each observation belongs to the cluster with the nearest mean (cluster centers or cluster centroid). Please not to be confused with K-nearest neighbors algorithm (KNN).

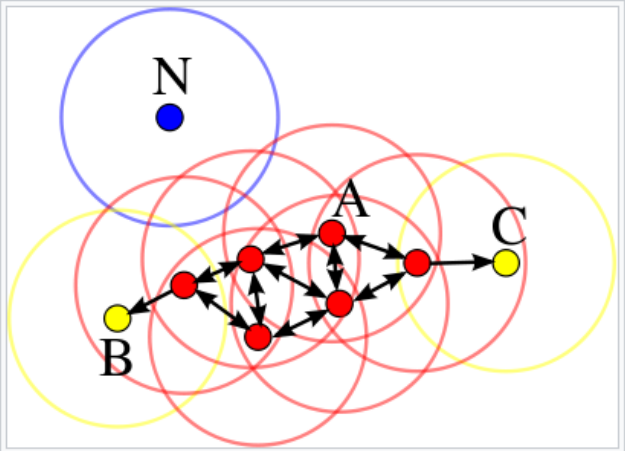
Centroid: In mathematics and physics, the centroid or geometric center of a plane figure is the arithmetic mean position of all the points in the figure.

Distance: Euclidean distance （欧氏距离）(perform normalization first)

Object: $min \sum_{i=1}^{K} \sum_{x \epsilon Ci} dist(Ci, x)^2$

DBSCAN

DBSCAN stands for density-based spatial clustering of applications with noise. It is able to find arbitrary shaped clusters and clusters with noise (i.e. outliers).



Principal Component Analysis

- Let's represent $\mathbf{x} \in R^N$ as a linear combination of an orthonormal set of N basis vectors $<v_1, v_2, \ldots, v_N>$ in R^N :

$$v_i^T v_j = \begin{cases} 1 & if \ i = j \\ 0 & otherwise \end{cases}$$

$$\mathbf{x} = \sum_{i=1}^{N} x_i v_i = x_1 v_1 + x_2 v_2 + \ldots + x_N v_N$$

$$where \quad x_i = \frac{\mathbf{x}^T v_i}{v_i^T v_i} = \mathbf{x}^T v_i$$

- PCA seeks to represent **x** in a new space of lower dimensionality using only K basis vectors (K < N)

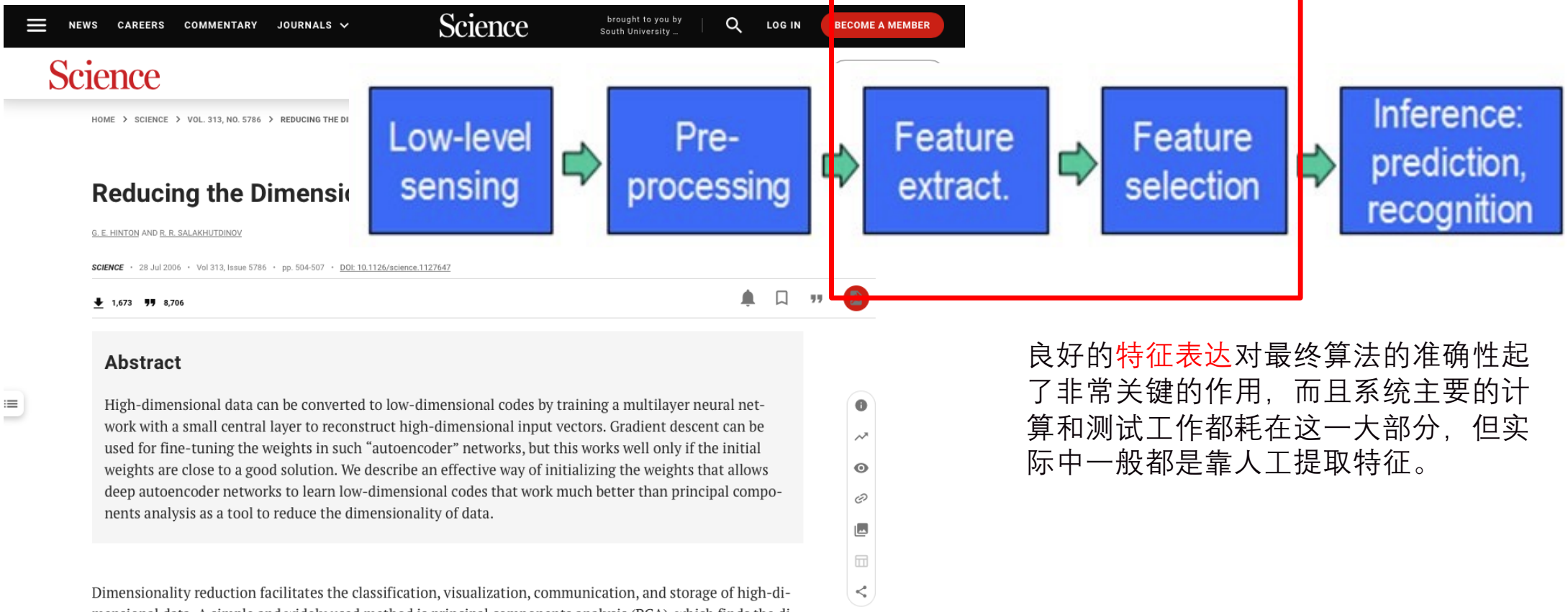
$$\hat{\mathbf{x}} = \sum_{i=1}^{K} y_i u_i = y_1 u_1 + y_2 u_2 + \ldots + y_K u_K$$

such that $\| \mathbf{x} - \hat{\mathbf{x}} \|$ is minimized

(i.e., minimize information loss)

注意与LDA的异同

Neural Network



良好的特征表达对最终算法的准确性起了非常关键的作用，而且系统主要的计算和测试工作都耗在这一大部分，但实际中一般都是靠人工提取特征。

Deep Learning=
Learning representations/features without being taught

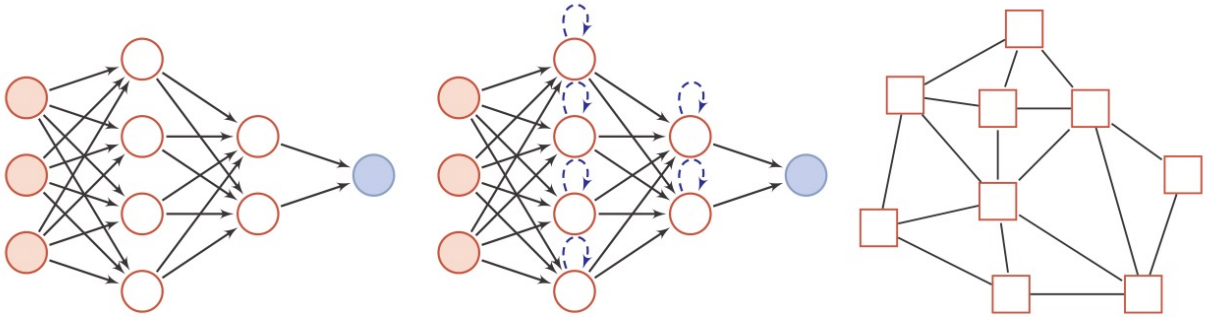
Bengio's survey paper Representation Learning:

A Review and New Perspectives emphasized this point.

Deep learning's main accomplishment is a universal data representation method with no need for domain specific knowledge

网络结构

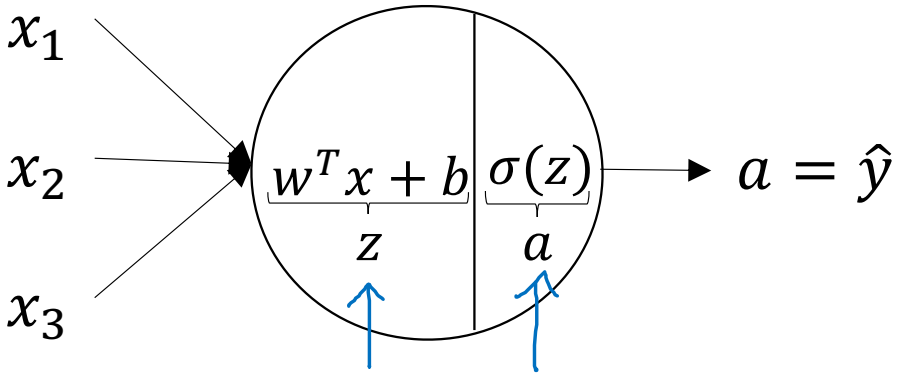
- 人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构。

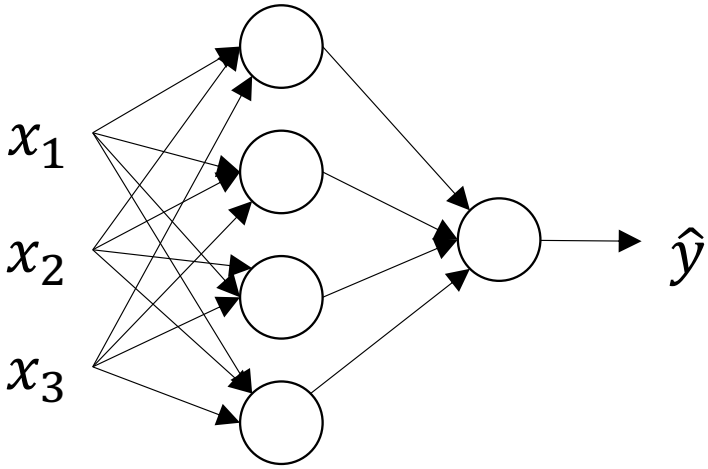


(a) 前馈网络　(b) 记忆网络　(c) 图网络

圆形节点表示一个神经元，方形节点表示一组神经元。

Neural Network Representation

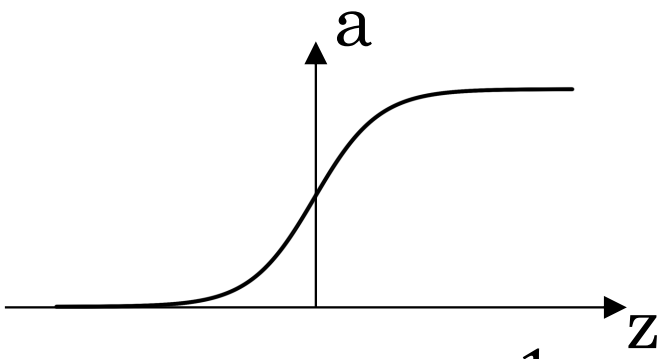




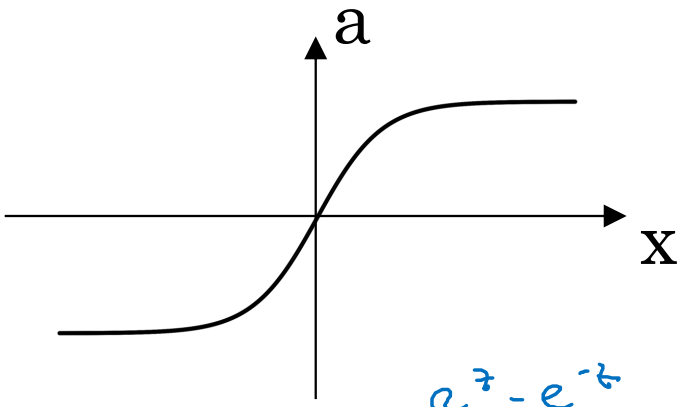
$$z = w^T x + b$$

$$a = \sigma(z)$$

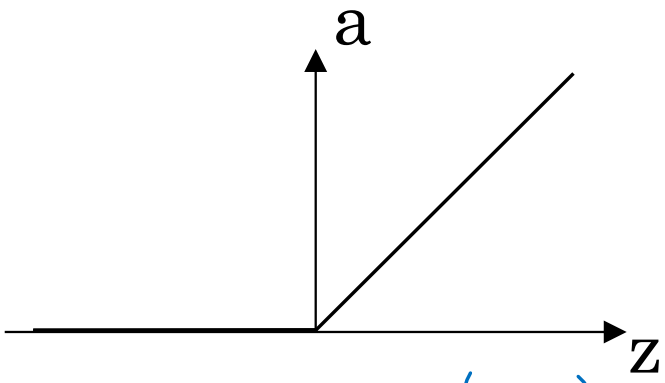
Pros and cons of activation functions



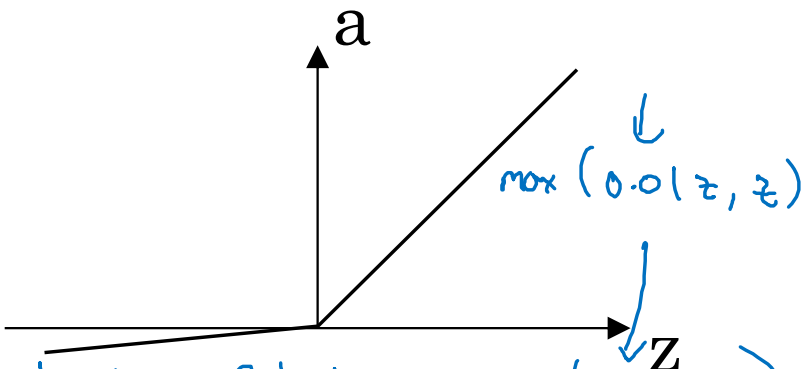
sigmoid: $a = \dfrac{1}{1+e^{-z}}$



tanh: $a = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$



ReLU $a = \max(0, z)$



Leaky ReLU $a = \max(0.01z, z)$

通用近似定理 (Universal Approximation Theorem)

定理 4.1 – 通用近似定理（Universal Approximation Theorem）[Cybenko, 1989, Hornik et al., 1989]： 令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数，$\mathcal{I}_d$ 是一个 d 维的单位超立方体 $[0,1]^d$，$C(\mathcal{I}_d)$ 是定义在 $\mathcal{I}_d$ 上的连续函数集合。对于任何一个函数 $f \in C(\mathcal{I}_d)$，存在一个整数 m，和一组实数 $v_i, b_i \in \mathbb{R}$ 以及实数向量 $\mathbf{w}_i \in \mathbb{R}^d$，$i = 1, \cdots, m$，以至于我们可以定义函数

$$F(\mathbf{x}) = \sum_{i=1}^{m} v_i \varphi(\mathbf{w}_i^{\mathrm{T}} \mathbf{x} + b_i), \tag{4.33}$$

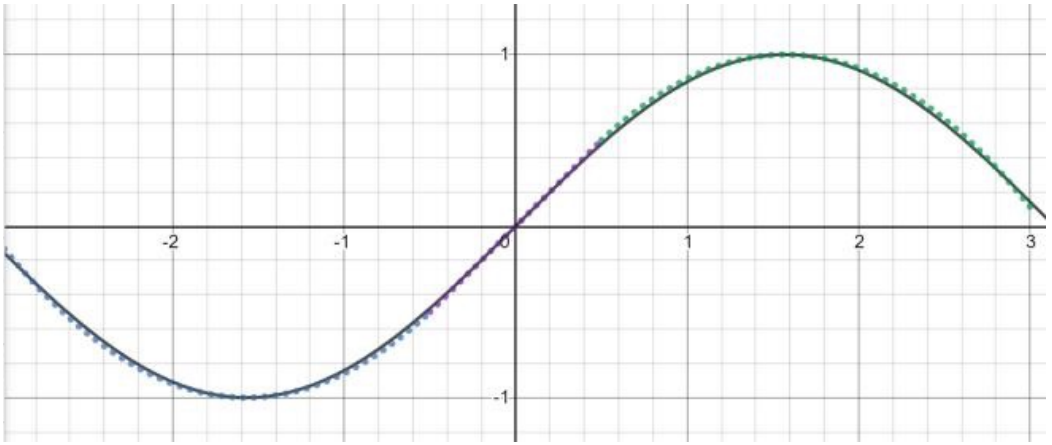
作为函数 f 的近似实现，即

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \forall \mathbf{x} \in \mathcal{I}_d. \tag{4.34}$$

其中 $\epsilon > 0$ 是一个很小的正数。

根据通用近似定理，对于具有线性输出层和至少一个使用“挤压”性质的激活函数的隐藏层组成的前馈神经网络，只要其隐藏层神经元的数量足够，它可以以任意的精度来近似任何从一个定义在实数空间中的有界闭集函数。

http://neuralnetworksanddeeplearning.com/chap4.html



1 $\sin(x)$

2 $\frac{1}{2.3}\left(x+\frac{\pi}{2}\right)^2 - 1 \{-3 < x < -0.5\}$

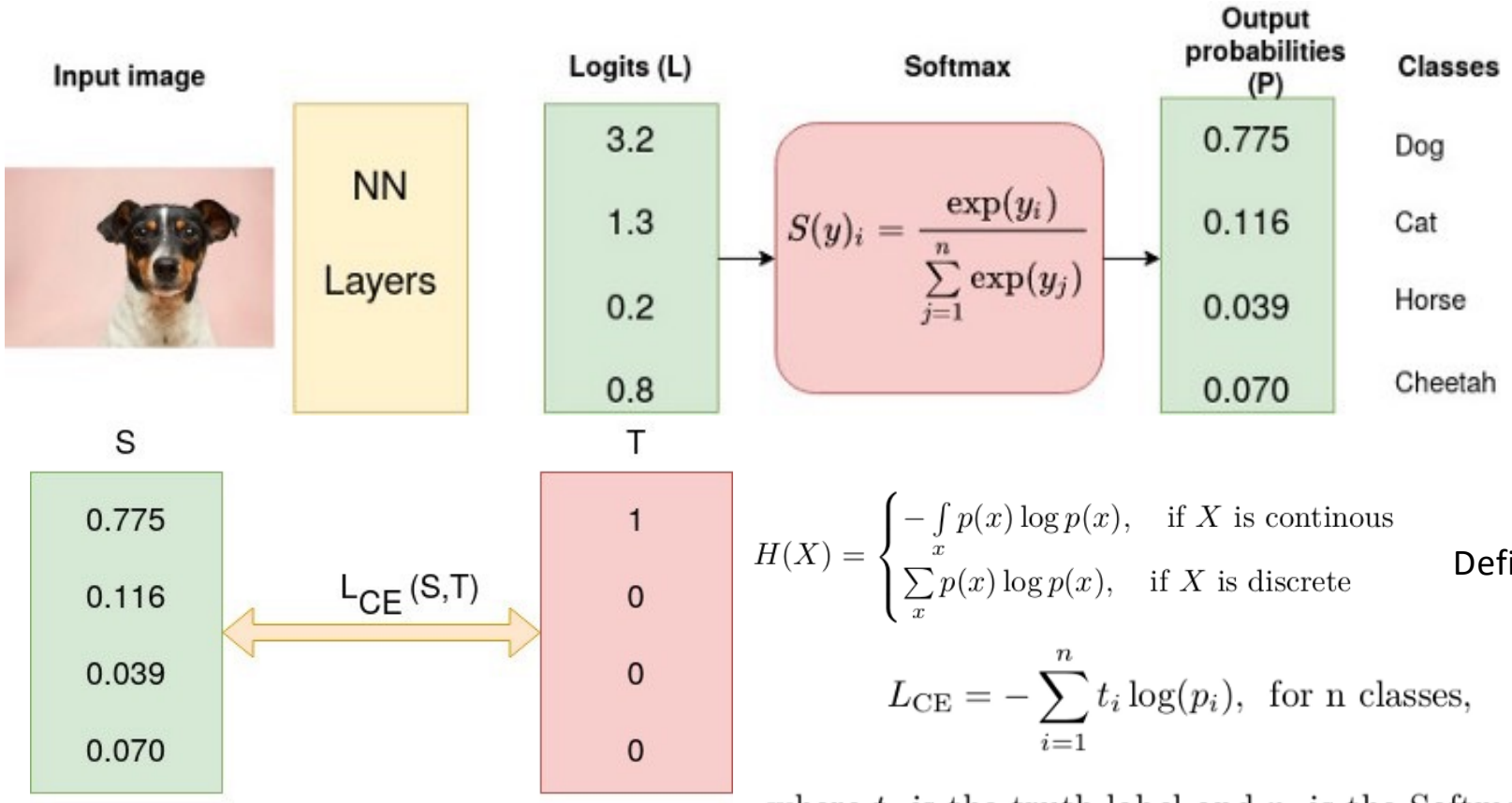
3 $\frac{1}{2.3} \cdot -\left(x-\frac{\pi}{2}\right)^2 + 1 \{0.5 < x < 3\}$

4 $y = x \{-0.5 < x < 0.5\}$

5

交叉熵损失函数 （Cross-Entropy Function）

分类问题主流算法



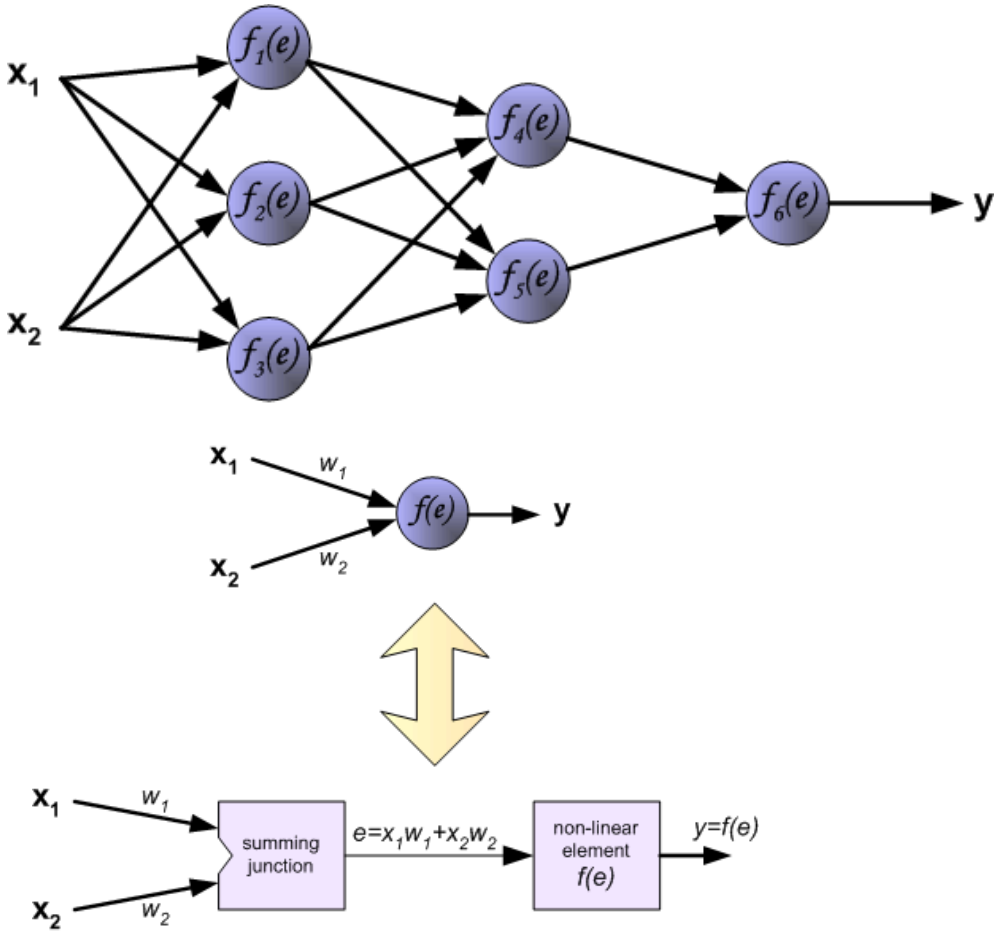
$$H(X) = \begin{cases} -\int_x p(x) \log p(x), & \text{if } X \text{ is continous} \\ \sum_x p(x) \log p(x), & \text{if } X \text{ is discrete} \end{cases}$$

Definition of Entropy

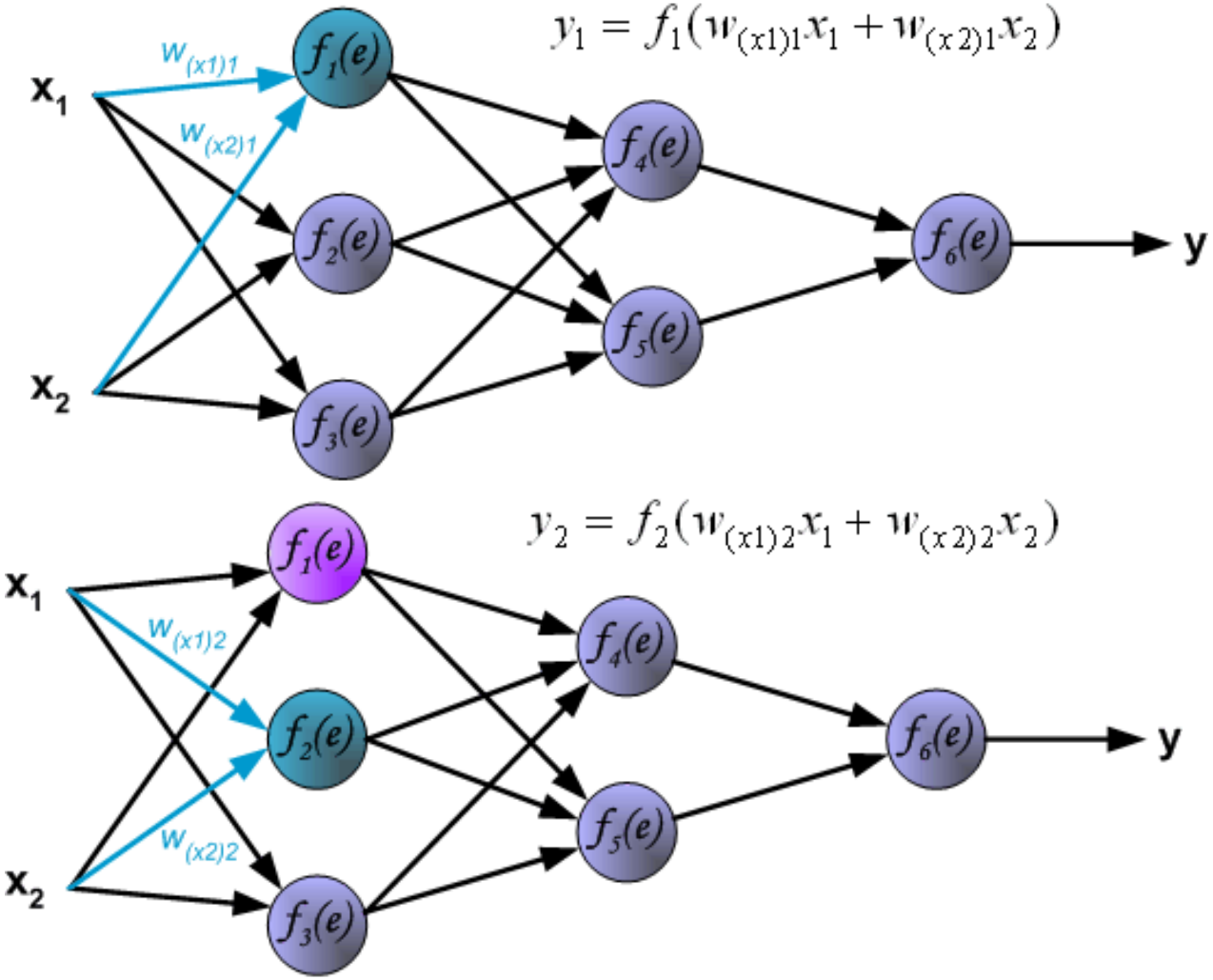
$$L_{\text{CE}} = -\sum_{i=1}^{n} t_i \log(p_i), \quad \text{for n classes,}$$

where t_i is the truth label and p_i is the Softmax probability for the i^{th} class.

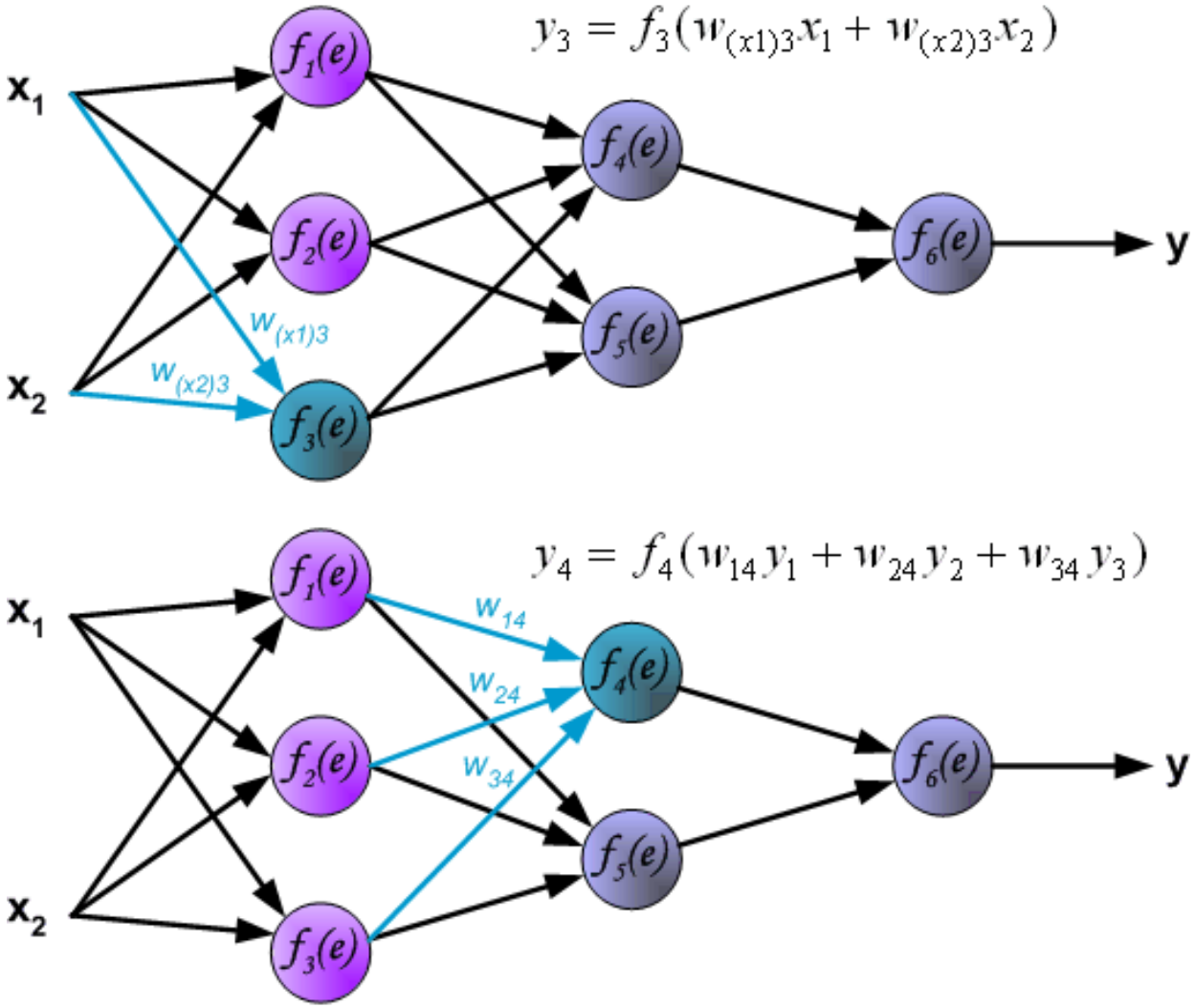
反向传播算法 (Backpropagation)



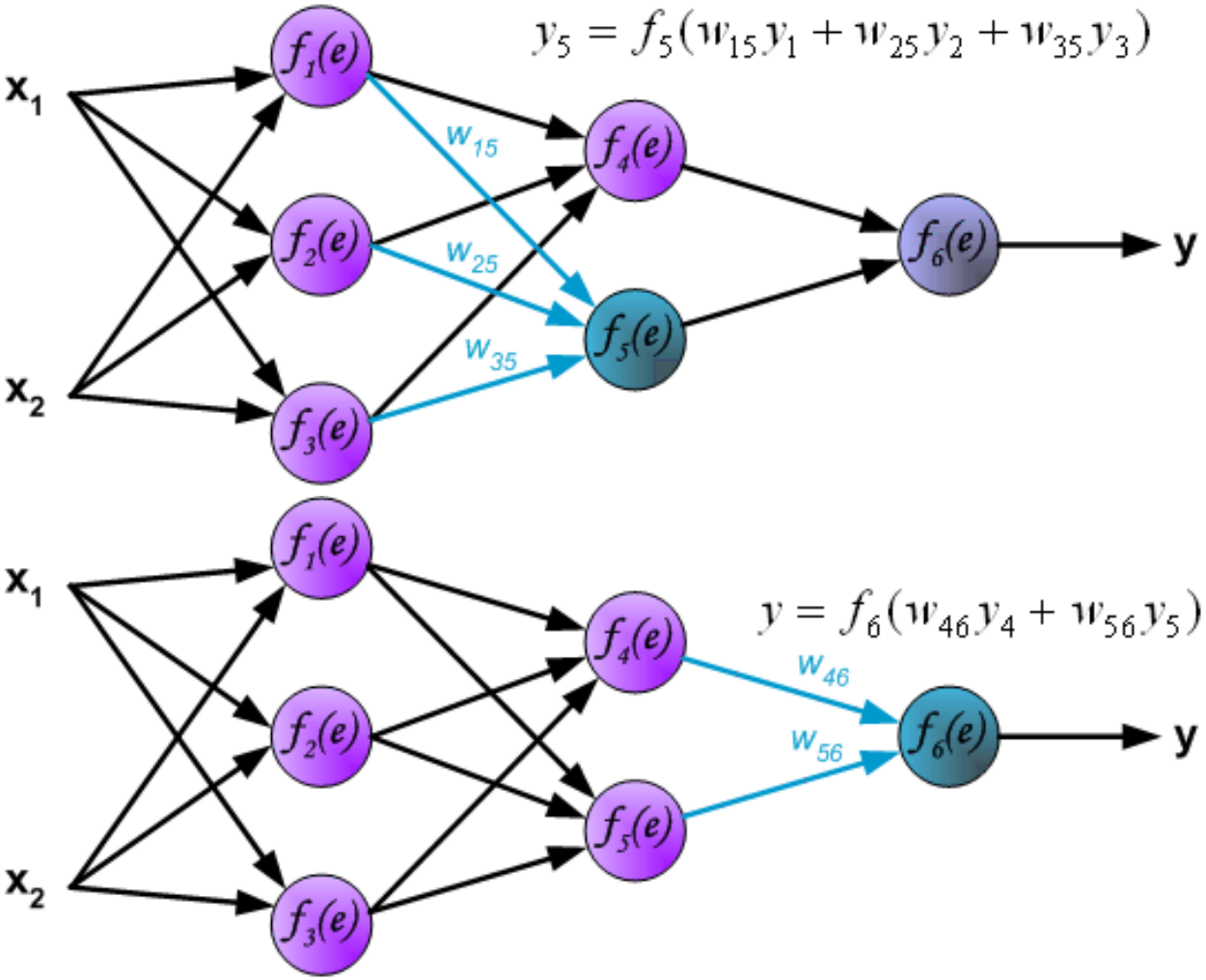
反向传播算法 (Backpropagation, Step-1)



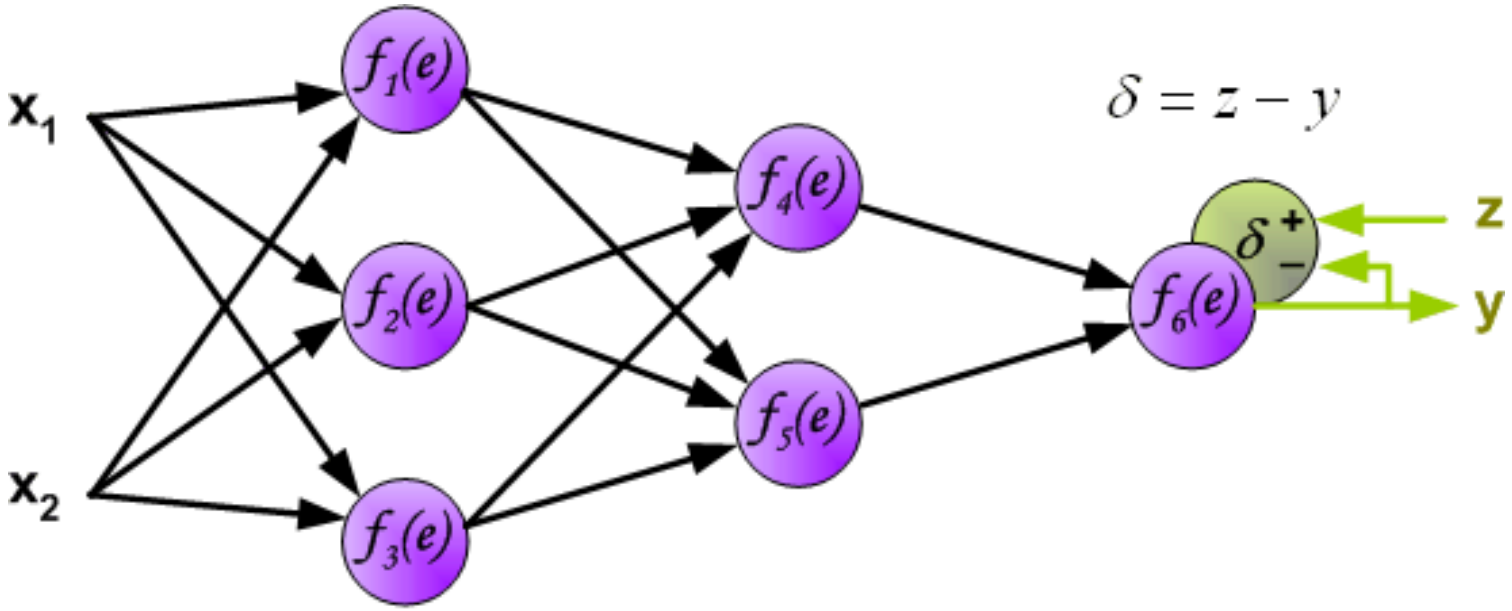
反向传播算法 (Backpropagation, Step-1)



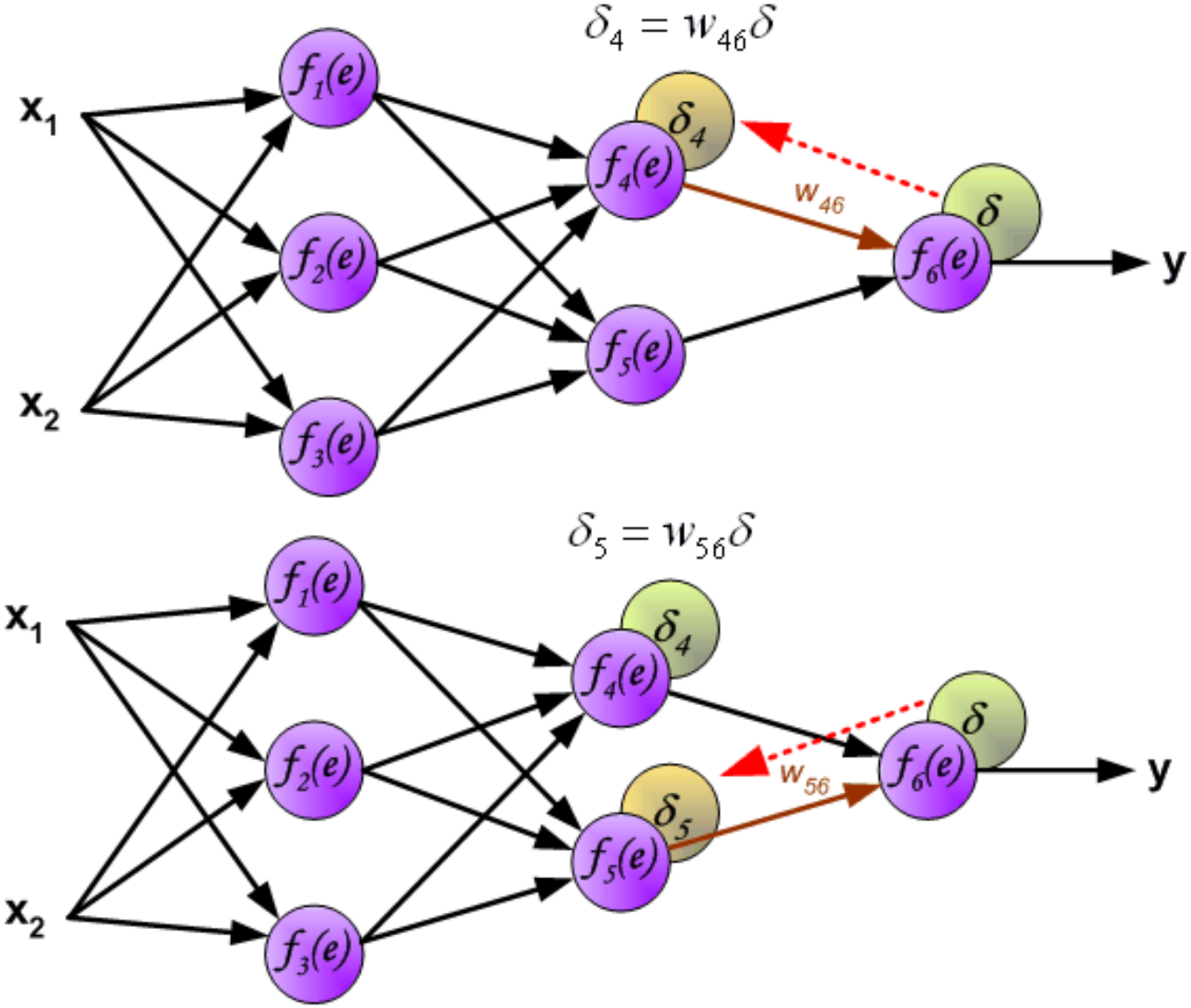
反向传播算法 (Backpropagation, Step-1)



反向传播算法 (Backpropagation, Step-1)

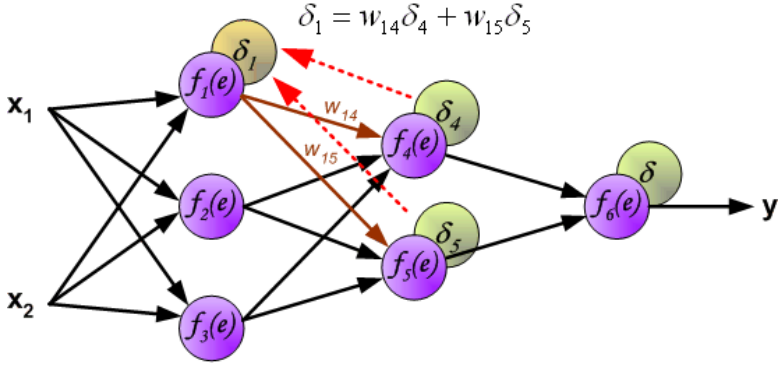


反向传播算法 (Backpropagation, Step-2)

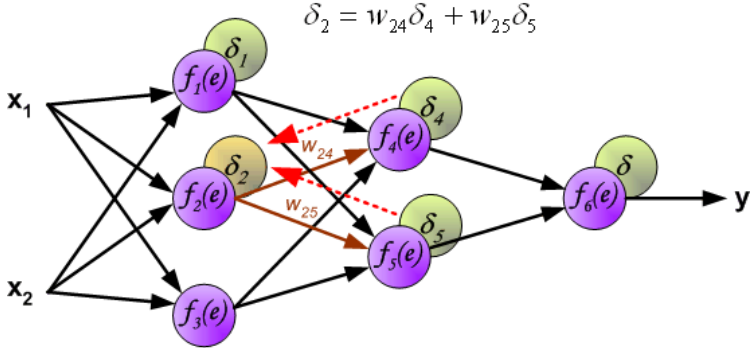


反向传播算法 (Backpropagation, Step-2)

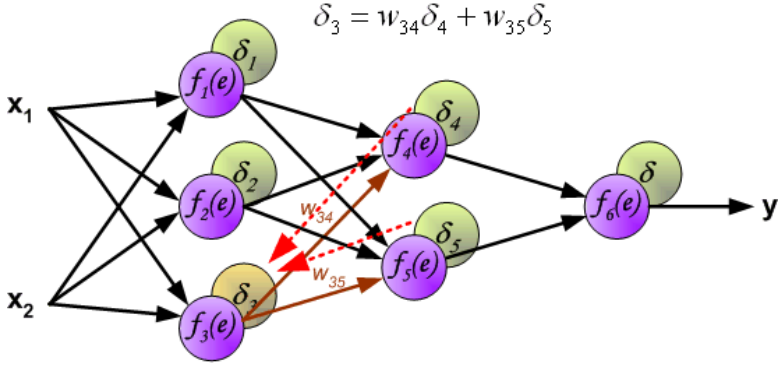
$\delta_1 = w_{14}\delta_4 + w_{15}\delta_5$



$\delta_2 = w_{24}\delta_4 + w_{25}\delta_5$



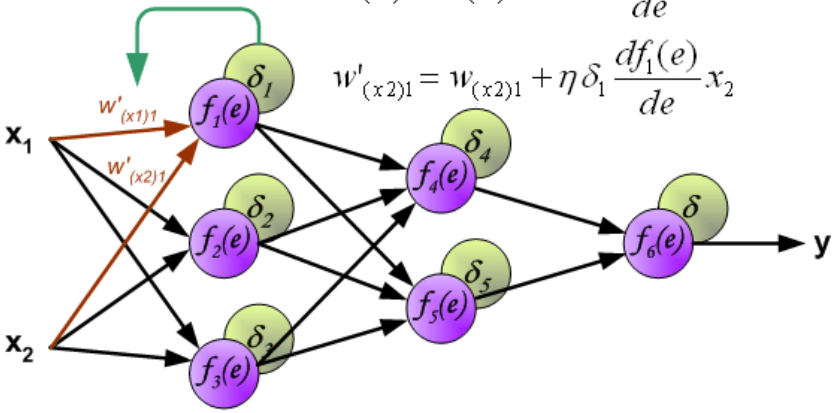
$\delta_3 = w_{34}\delta_4 + w_{35}\delta_5$



反向传播算法 (Backpropagation, Step-3)

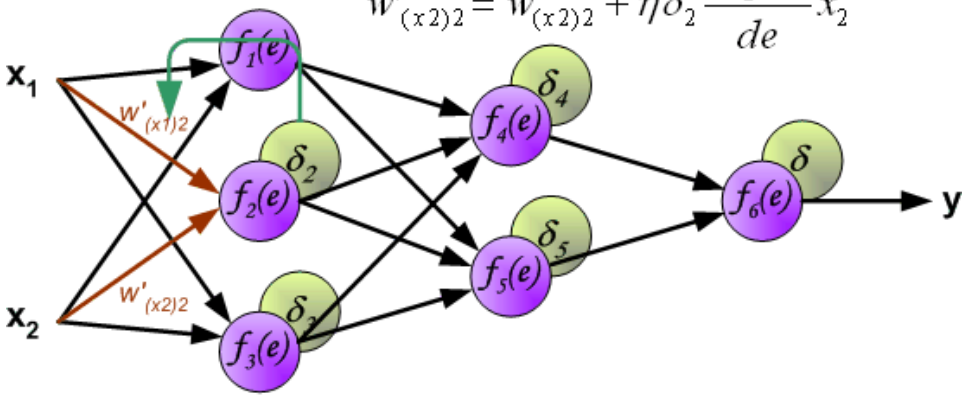
$$w'_{(x1)1} = w_{(x1)1} + \eta\delta_1 \frac{df_1(e)}{de} x_1$$

$$w'_{(x2)1} = w_{(x2)1} + \eta\delta_1 \frac{df_1(e)}{de} x_2$$



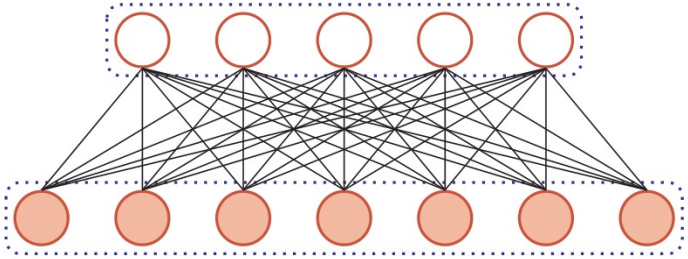
$$w'_{(x1)2} = w_{(x1)2} + \eta\delta_2 \frac{df_2(e)}{de} x_1$$

$$w'_{(x2)2} = w_{(x2)2} + \eta\delta_2 \frac{df_2(e)}{de} x_2$$



前馈全连接神经网络的缺点：

- 权重矩阵的参数非常多



- 局部不变性特征
  - 自然图像中的物体都具有局部不变性特征
    - 尺度缩放、平移、旋转等操作不影响其语义信息。
  - 全连接前馈网络很难提取这些局部不变特征

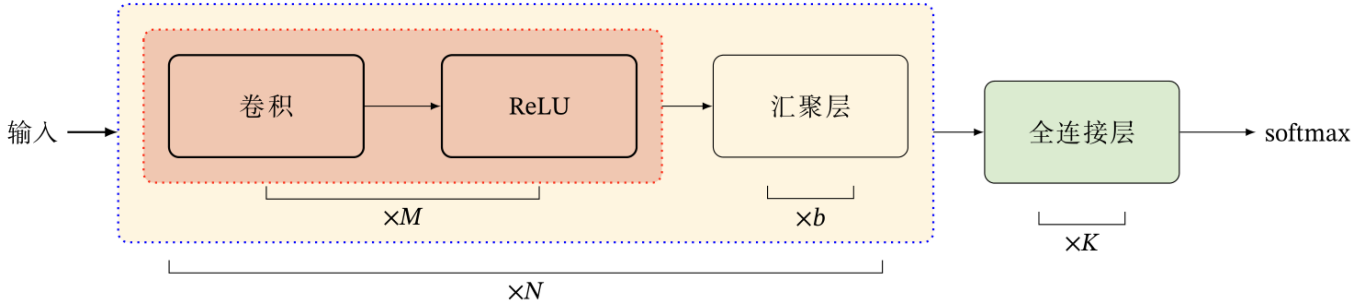
卷积神经网络

- 卷积神经网络（Convolutional Neural Networks，CNN）
  - 一种前馈神经网络
  - 受生物学上感受野（Receptive Field）的机制而提出的
    - 在视觉神经系统中，一个神经元的感受野是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元 (1981年David和Torsten诺贝尔奖)。

- 卷积神经网络有三个结构上的特性：
  - 局部连接 (local connection)
  - 权重共享 (weight sharing)
  - 空间或时间上的次采样 (sub-sampling)

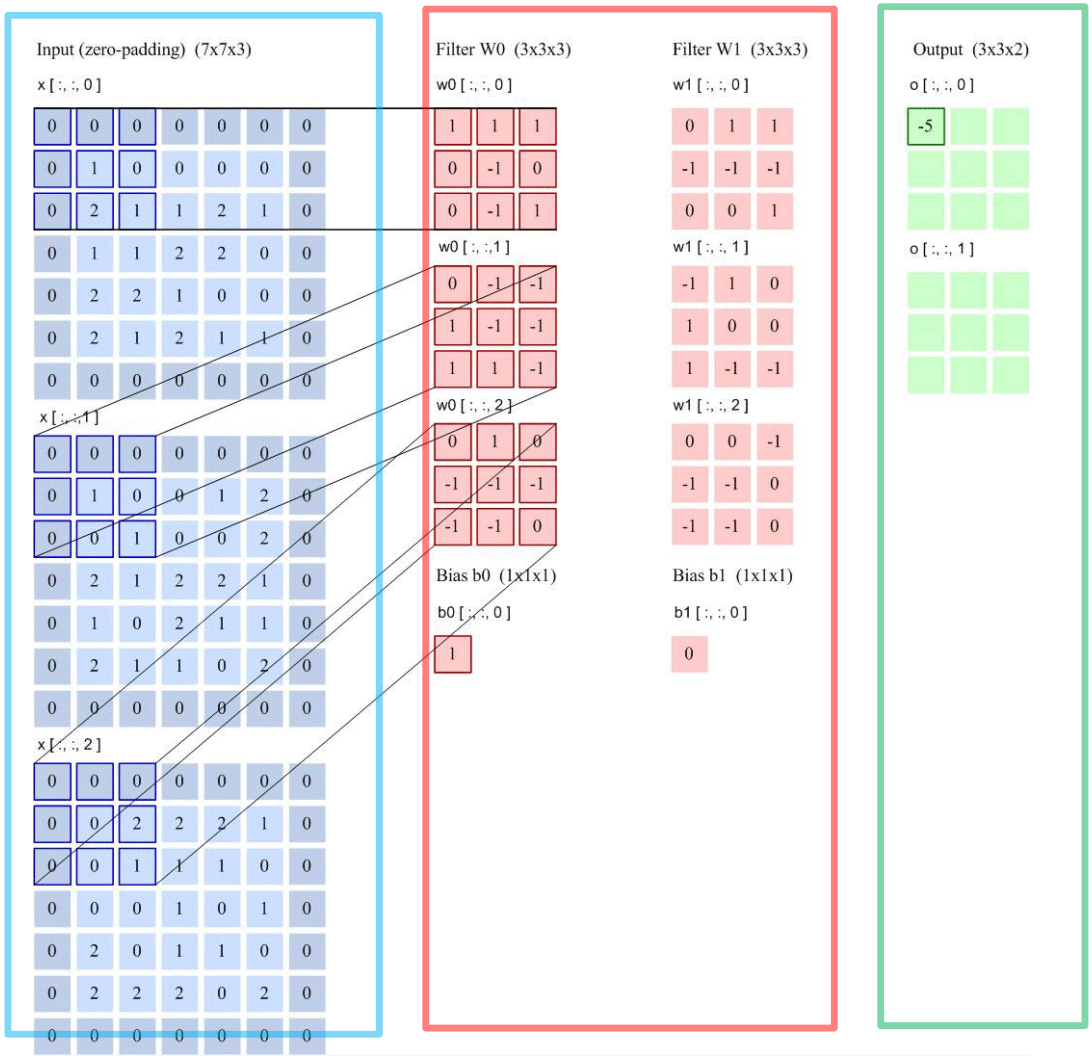
卷积神经网络结构

- 卷积网络是由卷积层、汇聚层、全连接层交叉堆叠而成。
  - 趋向于小卷积、大深度
  - 趋向于全卷积
- 典型结构



  - 一个卷积块为连续M 个卷积层和b个汇聚层（M通常设置为2 ～ 5，b为0或1）。一个卷积网络中可以堆叠N 个连续的卷积块，然后在接着K 个全连接层（N 的取值区间比较大，比如1 ～ 100或者更大；K一般为0 ～ 2）。

卷积神经网络运算



步长2
filter 3*3
filter个数6
零填充 1

其中，Output Volume中的o[:, :, 0]的输出结果1的计算方法为：

W0[:,:,0]* x[:,:,0]+W0[:,:,1]* x[:,:,1]+
W0[:,:,2]* x[:,:,2]+b
*为卷积操作

卷积神经网络参数个数

```
model = tf.keras.Sequential(
    [
    tf.keras.layers.Conv2D(32, (3,3), padding='same', activation="relu",input_shape=(32, 32, 3)),
    tf.keras.layers.MaxPooling2D((2, 2), strides=2),

    tf.keras.layers.Conv2D(64, (3,3), padding='same', activation="relu"),
    tf.keras.layers.MaxPooling2D((2, 2), strides=2),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(10, activation="softmax")
]
```

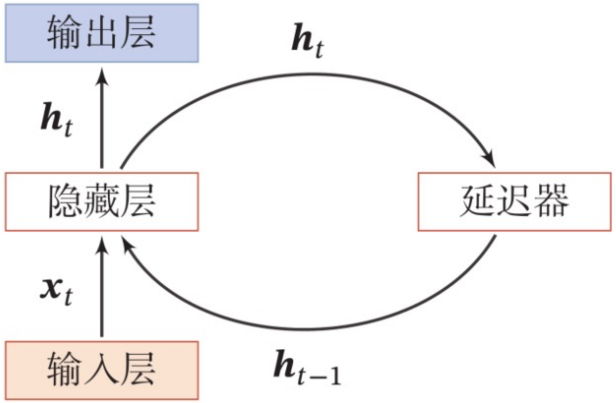
循环神经网络 (Recurrent Neural Network, RNN)

- 循环神经网络通过使用带自反馈的神经元，能够处理任意长度的时序数据。

$$\boldsymbol{h}_t = f(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t)$$

活性值

状态



- 循环神经网络比前馈神经网络更加符合生物神经网络的结构。
- 循环神经网络已经被广泛应用在语音识别、语言模型以及自然语言生成等任务上

- 状态更新：

$$\boldsymbol{h}_t = f(\boldsymbol{U}\boldsymbol{h}_{t-1} + \boldsymbol{W}\boldsymbol{x}_t + \boldsymbol{b})$$

- 一个完全连接的循环网络是任何非线性动力系统的近似器 。

定理 6.1 – 循环神经网络的通用近似定理 [Haykin, 2009]： 如果一个完全连接的循环神经网络有足够数量的 sigmoid 型隐藏神经元，它可以以任意的准确率去近似任何一个非线性动力系统

$$\boldsymbol{s}_t = g(\boldsymbol{s}_{t-1}, \boldsymbol{x}_t), \tag{6.10}$$

$$\boldsymbol{y}_t = o(\boldsymbol{s}_t), \tag{6.11}$$

其中 $\boldsymbol{s}_t$ 为每个时刻的隐状态，$\boldsymbol{x}_t$ 是外部输入，$g(\cdot)$ 是可测的状态转换函数，$o(\cdot)$ 是连续输出函数，并且对状态空间的紧致性没有限制.

- 循环神经网络在时间维度上非常深！
  - 梯度消失或梯度爆炸
- 如何改进？
  - 梯度爆炸问题
    - 权重衰减
    - 梯度截断
  - 梯度消失问题
    - 改进模型

IEEE TRANSACTIONS ON NEURAL NETWORKS, VOL. 5, NO. 2, MARCH 1994

Learning Long-Term Dependencies with Gradient Descent is Difficult

Yoshua Bengio, Patrice Simard, and Paolo Frasconi, *Student Member, IEEE*

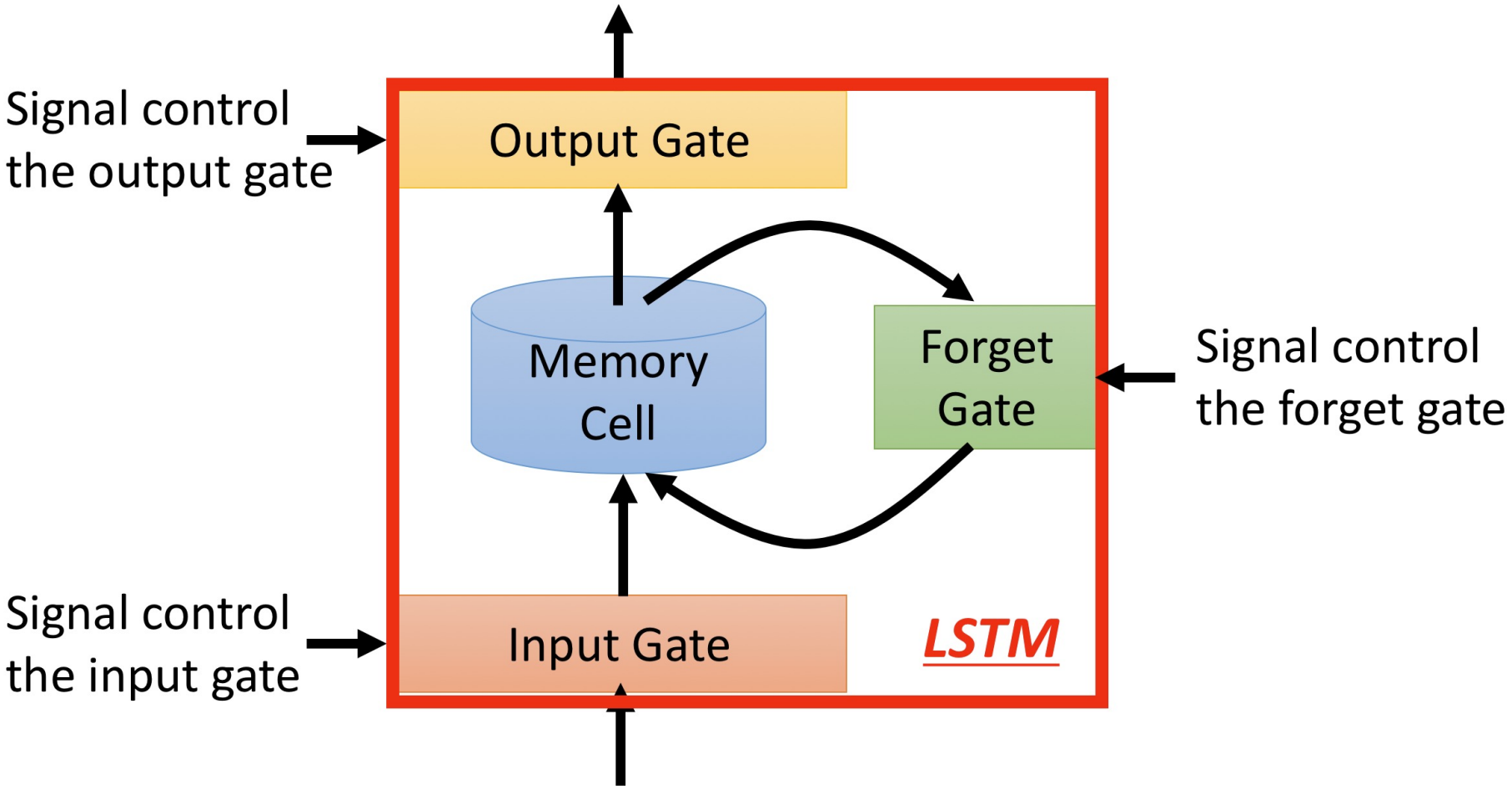
***Abstract*— Recurrent neural networks can be used to map input sequences to output sequences, such as for recognition, production or prediction problems. However, practical difficulties have been reported in training recurrent neural networks to perform tasks in which the temporal contingencies present in the input/output sequences span long intervals. We show why gradient based learning algorithms face an increasingly difficult problem as the duration of the dependencies to be captured increases. These results expose a trade-off between efficient learning by gradient descent and latching on information for long periods. Based on an understanding of this problem, alternatives to standard gradient descent are considered.**

I. Introduction

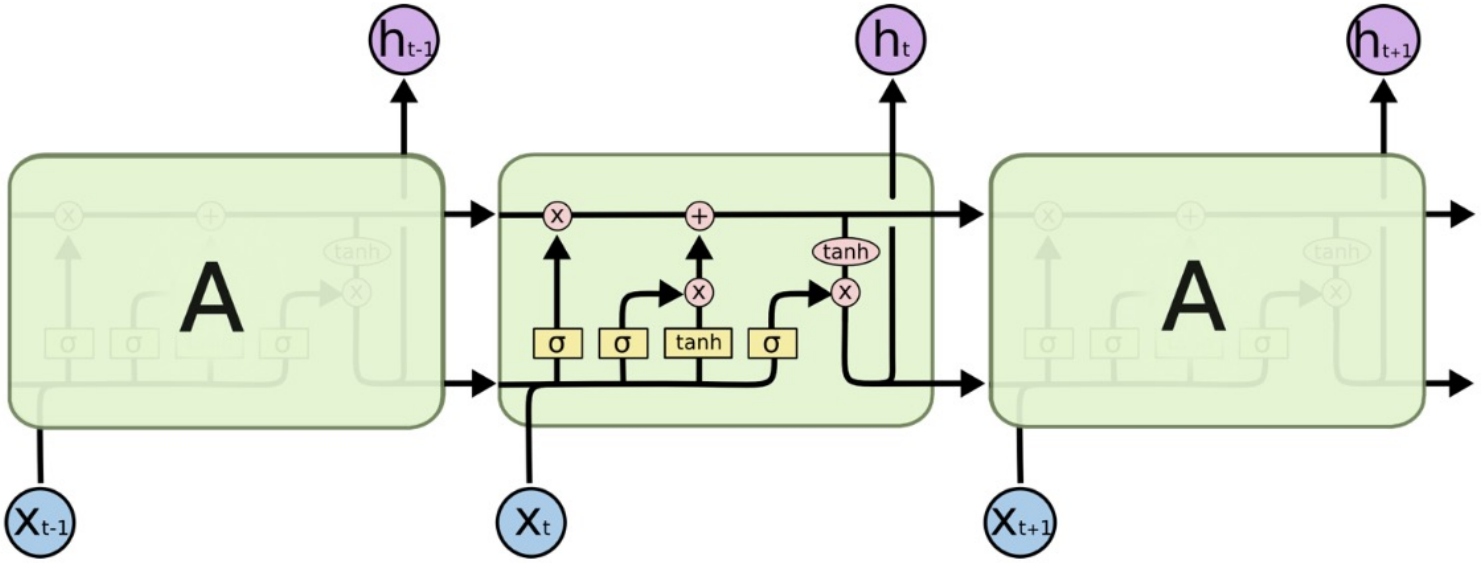
WE ARE INTERESTED IN training recurrent neural networks to map input sequences to output sequences, for applications in sequence recognition, production, or time-series prediction. All of the above applications require a system that will store and update context information; i.e., information computed from the past inputs and useful to produce desired outputs. Recurrent neural networks are well suited for those tasks because they have an internal state that can represent context information. The cycles in the graph of a recurrent network allow it to keep information about past inputs for an a fully connected recurrent network) but are local in time; i.e., they can be applied in an on-line fashion, producing a partial gradient after each time step. Another algorithm was proposed [10], [18] for training constrained recurrent networks in which dynamic neurons—with a single feedback to themselves—have only incoming connections from the input layer. It is local in time like the forward propagation algorithms and it requires computation only proportional to the number of weights, like the back-propagation through time algorithm. Unfortunately, the networks it can deal with have limited storage capabilities for dealing with general sequences [7], thus limiting their representational power.

A task displays long-term dependencies if prediction of the desired output at time t depends on input presented at an earlier time $\tau \ll t$. Although recurrent networks can in many instances outperform static networks [4], they appear more difficult to train optimally. Earlier experiments indicated that their parameters settle in sub-optimal solutions that take into account short-term dependencies but not long-term dependencies [5]. Similar results were obtained by Mozer [19]. It was found that back-propagation was not sufficiently powerful to discover contingencies spanning long temporal

长短期记忆神经网络 (Long Short-Term Memory, LSTM)



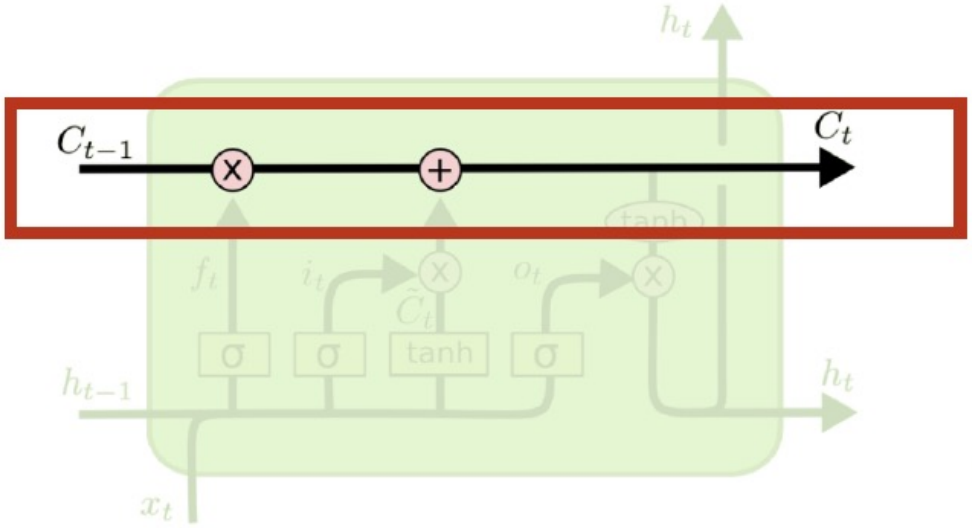
LSTM 属于一种特殊的RNN，Hochreiter & Schmidhuber (1997)

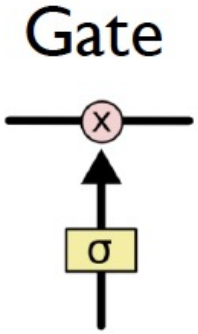


The repeating module in an LSTM contains four interacting layers.

LSTM的核心思想

关键部分: cell state





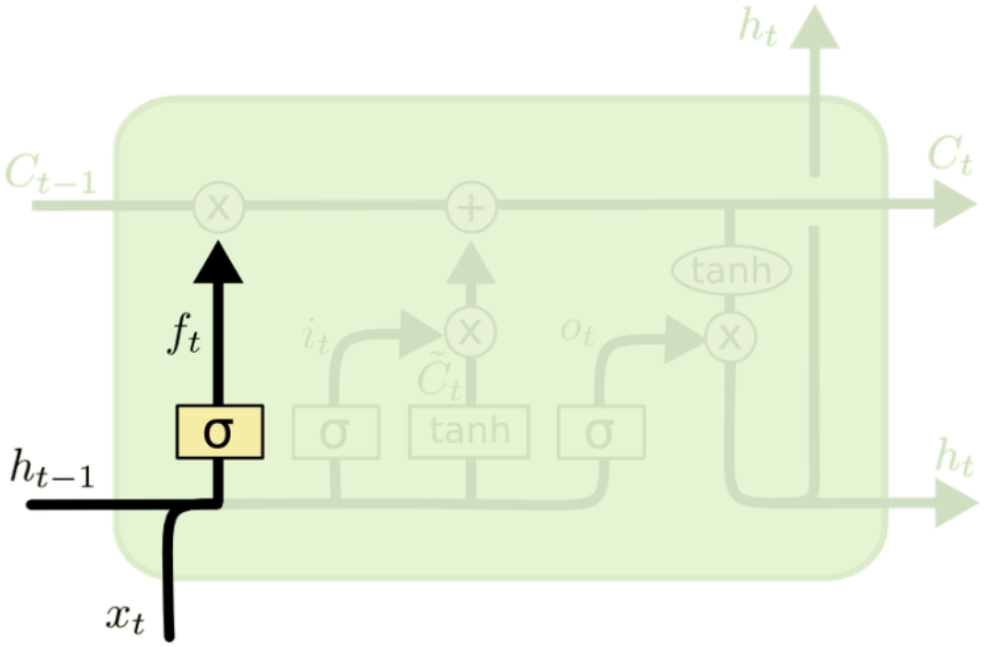
0: "let nothing through"

1: "let everything through"

LSTM用三种门来控制cell state

Forget gate

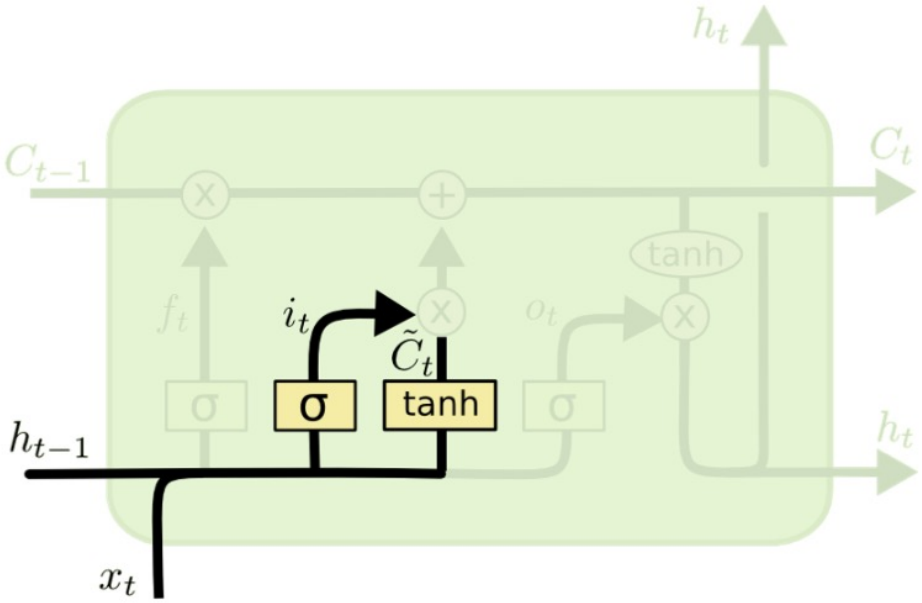
Forget门用于控制哪些信息该遗忘？哪些信息该记忆？



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Input gate门包括两部分

A perceptron layer：确定输入中是否有感兴趣的地方？
A gate：确定是否进行记忆？

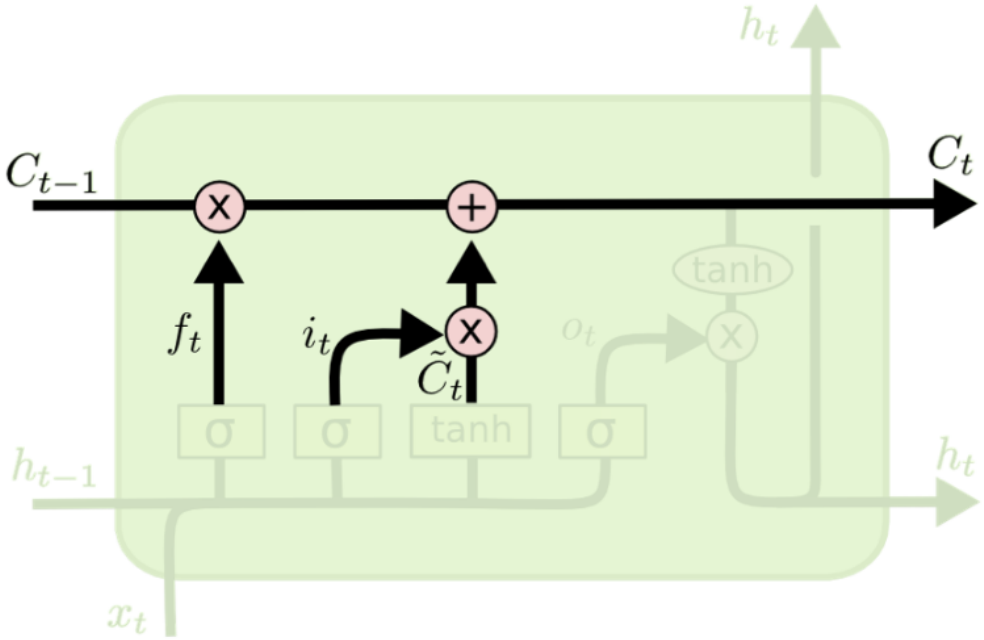


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

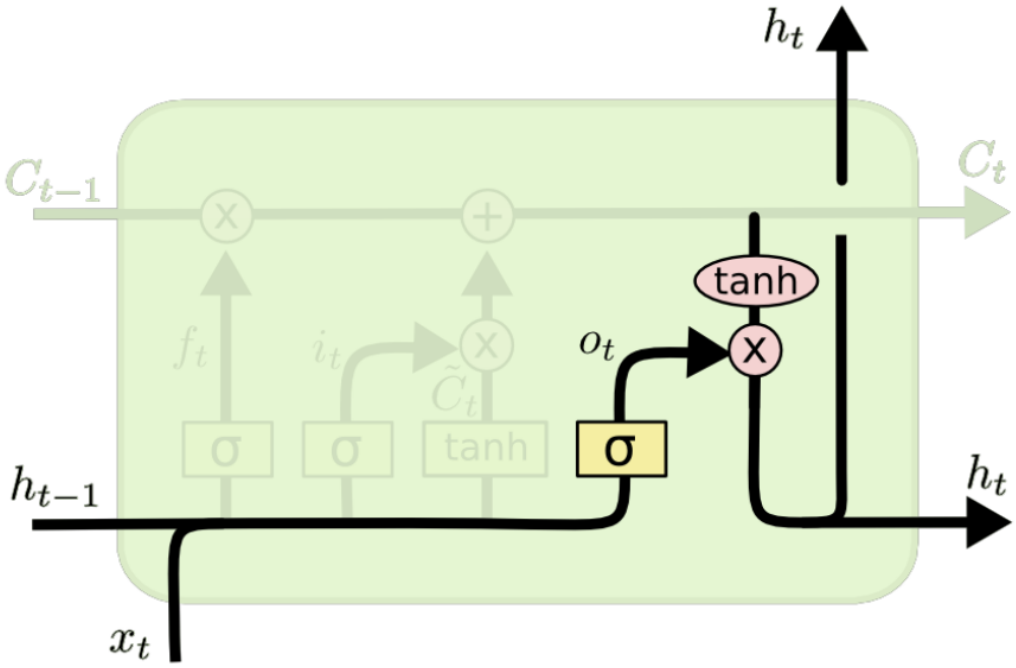
Input gate门包括两部分

A perceptron layer：确定输入中是否有感兴趣的地方？
A gate：确定是否进行记忆？
如果需要记忆，加入cell state



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output gate用输入和记忆内容计算神经元输出



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

LSTM的神经元个数

$$\begin{pmatrix} a_1^1 & a_1^2 & \cdots & a_1^5 \\ a_2^1 & a_2^2 & \cdots & a_2^5 \\ \vdots & \vdots & \ddots & \vdots \\ a_{13}^1 & a_{13}^2 & \cdots & a_{13}^5 \end{pmatrix}$$

Shape=(13,5),时间步是13,
每个时间步的特征长度是5

```
from keras.layers import LSTM
from keras.models import Sequential

time_step=13
featrue=5
hidenfeatrue=10

model=Sequential()
model.add( LSTM(hidenfeatrue,input_shape=(time_step,featrue)))
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
lstm_8 (LSTM)                (None, 10)                640
=================================================================
Total params: 640
Trainable params: 640
Non-trainable params: 0
_________________________________________________________________
```

Recipe of Deep Learning

- 网络优化
  - 神经网络优化的特点
- 优化算法
  - 优化算法改进
  - 动态学习率
  - 梯度估计修正
- 其他优化
  - 参数初始化
  - 数据预处理
  - 逐层规范化
  - 超参数优化

- 网络正则化
  - ℓ_1和ℓ_2正则化（跳过前几轮）
  - Dropout
  - Early-stop
  - 数据增强

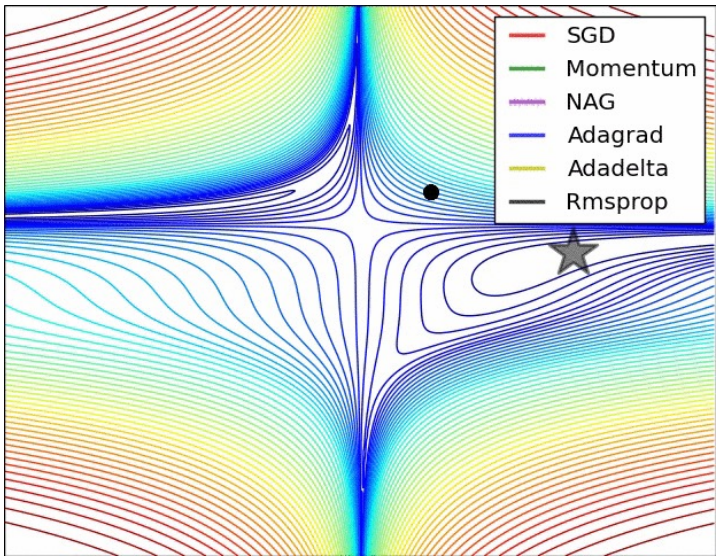
Recipe of Deep Learning

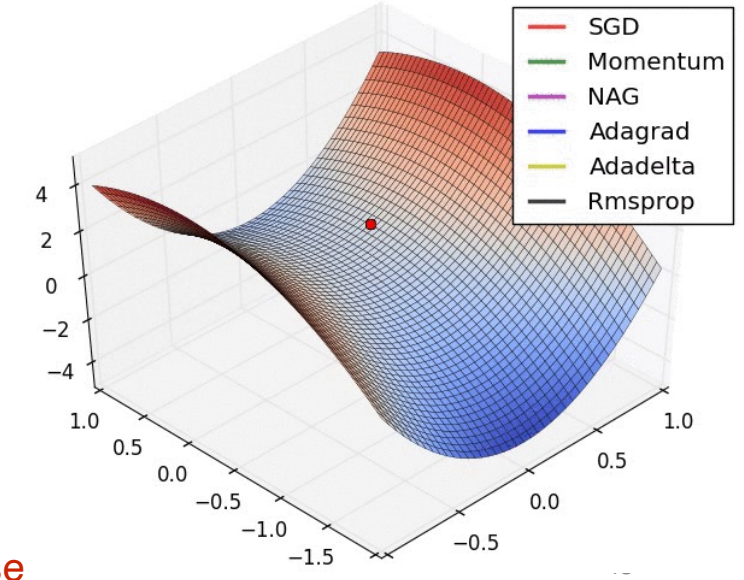
如何设定学习率?

- Gradient descent optimization algorithms
  - Adagrad
  - Adadelta
  - RMSprop
  - Momentum
  - Nesterov accelerated gradient
  - Adam
  - AdaMax
  - Aadam

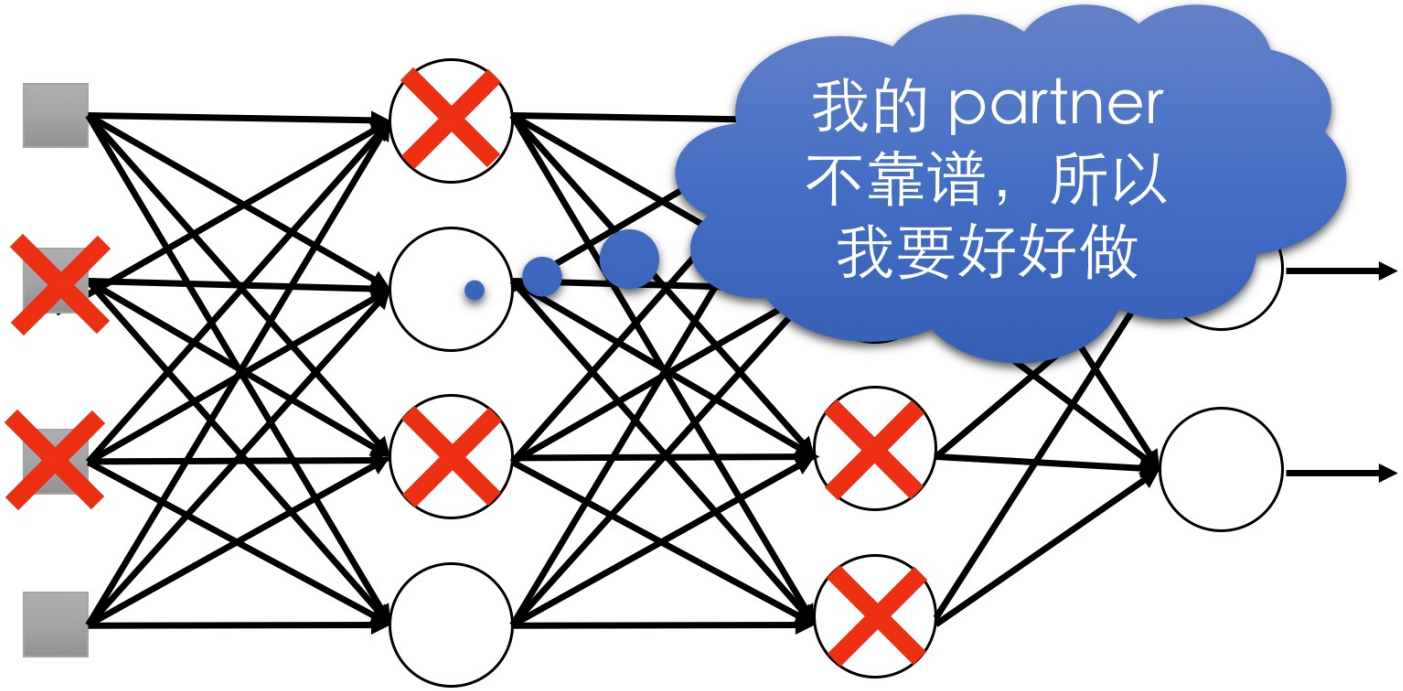
- Which optimizer to choose?

http://ruder.io/optimizing-gradient-descent/index.html#whichoptimizertochoose





Dropout的设计直觉



- 当团队合作（Training）时，如果每个人都指望合作伙伴来完成这项工作，最终什么都做不成
- 但是当你知道你的合作伙伴随时有可能退出时，你会尽力好好做
- 当测试时，实际上没有一个人退出，所以最终会取得好成绩

让AI成为科学发现的伙伴，而不是思考的替代品